

MAR ATHANASIOUS COLLEGE OF ENGINEERING
KOTHAMANGALAM
(Government Aided Autonomous Institution)



Department of Computer Applications

Main Project Report

**AR-SPACE : VIRTUAL INTERIOR DESIGN USING
AUGMENTED REALITY**

Done by

ANOOP KRISHNAN

Reg No: MAC24MCA-2009

Under the guidance of

Prof. Manu John

2024–2026

**MAR ATHANASIOUS COLLEGE OF ENGINEERING
KOTHAMANGALAM
(Government Aided Autonomous Institution)**

CERTIFICATE



AR-Space : Virtual Interior Design using Augmented Reality

Certified that this is the bonafide record of project work done by

Anoop Krishnan

Reg No: MAC24MCA-2009

during the fourth semester, in partial fulfilment of requirements for
the award of the degree,

Master of Computer Applications

of

APJ Abdul Kalam Technological University, Thiruvananthapuram

Prof. Manu John

Faculty Guide

Prof. Biju Skaria

Head of the Department

Prof. Shinu S Kurian

Project Coordinator

External Examiner

Acknowledgement

With heartfelt gratitude, I extend my deepest thanks to the Almighty for his unwavering grace and blessings that have made this journey possible. May his guidance continue to illuminate my path in the years ahead.

I am immensely thankful to Prof. Biju Skaria, head of the department of Computer Applications, Prof. Shinu S Kurian our dedicated project coordinator and Prof. Manu John my project guide for their invaluable guidance and timely advice, which played a pivotal role in shaping this project. Their guidance, constant supervision, and provision of essential information were instrumental in the successful completion of the project.

I extend my profound thanks to all the professors in the department and the entire staff at Mar Athanasius College of Engineering for their unwavering support and inspiration throughout my academic journey. My sincere appreciation goes to my beloved parents, whose guidance has been a beacon in every step of our path.

I am also grateful to my friends and individuals who generously shared their expertise and assistance, contributing significantly to the fulfillment of this endeavor.

Anoop Krishnan

Abstract

The rapid advancement of Augmented Reality (AR) technology has opened new possibilities in the field of interior design by enabling users to visualize virtual objects within real-world environments. This project presents the development of a mobile-based AR application that allows users to design and customize interior spaces by placing virtual furniture in real time.

The application is developed using the Unity game engine integrated with AR Foundation, enabling cross-platform AR capabilities. It utilizes ARCore for Android-based environment tracking, plane detection, and light estimation. The system supports accurate surface detection, allowing users to place 3D furniture models realistically within their surroundings. The user interface is designed to be intuitive, enabling users to select room types, browse furniture categories, and interact with objects through gestures such as move, rotate, and scale.

To enhance functionality, the application incorporates cloud based services for storing and retrieving design layouts, ensuring accessibility across sessions. The backend integration is designed to support data persistence and potential scalability for multi-user access. Additionally, optimized 3D models and efficient rendering techniques are used to ensure smooth performance on mobile devices.

The system also includes features such as design saving, design sharing, and real-time visualization, which improve user experience and usability. By allowing users to experiment with different layouts before implementation, the application helps reduce uncertainty and supports better decision-making in interior design.

This project demonstrates how AR technology, combined with modern development frameworks and cloud integration, can be effectively utilized to create a user-friendly and practical solution for interior design. It offers an immersive and interactive experience that saves time, enhances creativity, and increases user confidence in design choices.

List of Tables

2.1	Reference paper 1 summary	3
2.2	Reference paper 2 summary	5
2.3	Reference paper 3 summary	7
2.4	Literature review summary	9

List of Figures

3.1	Project Pipeline	13
4.1	AR Foundation architecture diagram	18
4.2	System workflow diagram	24
6.1	AR Furniture Placement using Raycasting	41
6.2	Object Selection	42
6.3	Drag and Reposition of Furniture Object	42
6.4	Rotation of Furniture using Multi-Touch Gesture	43
6.5	Double-Tap Deletion of Furniture Object	43
6.6	Registering and Maintaining Placed Furniture Objects	44
6.7	Dynamic Cost Addition for Placed Furniture	45
6.8	Cost Removal upon Deleting Furniture	46
6.9	Budget Validation and Warning Display	47
6.10	Unique ID Generation and Layout Data Collection	48
6.11	Screenshot Capture and Processing	49
6.12	Saving Layout Data and Screenshot to Cloud	49
6.13	Retrieving Saved Designs from Cloud	50
6.14	Deleting Saved Design from Cloud	51
6.15	Loading and Displaying Saved Design Slots	51
6.16	Room Selection and UI Panel Activation	52
6.17	Furniture Selection and Price Passing from UI	53
6.18	Budget Input Parsing and Initialization	54
7.1	Room Type Selection and Budget Setup Interface	55
7.2	Furniture Recommendation Panel.....	57
7.3	AR Design Workspace Interface	59
7.4	Saved Design Reload Interface	61
10.1	Git history	66

CONTENTS

Acknowledgement	i
Abstract	ii
List of Tables	iii
List of Figures	iv
1 INTRODUCTION	1
2 SUPPORTING LITERATURE	2
2.1 Literature Review	2
2.2 Literature Review Summary	7
2.3 Findings and Proposal	9
3 SYSTEM ANALYSIS	10
3.1 Analysis of Existing Systems	10
3.2 Limitations of Standard AR Placement	11
3.3 Proposed Solution: User-Relative Coordinate Mapping	11
3.4 Project Pipeline	13
3.4.1 AR Foundation & ARCore Setup	13
3.4.2 AR Session and Plane Detection Configuration	14
3.4.3 Environment Scanning and Surface Validation	14
3.4.4 3D Furniture Model Integration	14
3.4.5 User Interaction Implementation	14
3.4.6 AR Placement & Real-Time Visualization	14
3.4.7 Cloud Integration for Saving Designs	14
3.5 Feasibility Analysis	15
3.5.1 Technical Feasibility	15
3.5.2 Economical Feasibility	15
3.5.3 Operational Feasibility	15
3.6 System Environment	16
3.6.1 Software Environment	16
3.6.2 Hardware Environment	16
4 SYSTEM DESIGN	18
4.1 System Architecture	18
4.2 Core Functional Components	19

4.2.1	ARFurniturePlacer – AR Placement Module	19
4.2.2	FurnitureManager – Furniture Management Module	20
4.2.3	CostManager – Cost Estimation Module	20
4.2.4	CloudSaveManager – Cloud Save Module	21
4.2.5	SaveandLoadUIManager – Save and Load UI Module	21
4.2.6	RoomSetupManager – Room Setup Module	21
4.2.7	LayoutAlignmentController – Layout Alignment Module	22
4.3	AR Environment Setup	22
4.3.1	AR Session	22
4.3.2	AR Session Origin	23
4.3.3	AR Camera	23
4.3.4	AR Plane Manager	23
4.3.5	AR Raycast Manager	23
4.4	Application Workflow and User Interaction Diagram	24
5	METHODOLOGY	26
5.1	AR Placement Methodology	26
5.2	Camera-Relative Coordinate Translation	27
5.3	Budget Management and Conditional Validation Logic	28
5.4	Cloud Persistence Method: Data Storage and Retrieval	30
5.5	Screenshot Capture, Buffer Management, and UI Handling	33
5.6	Layout Reload Methodology	35
5.7	Layout Alignment and Interaction Refinement	37
6	IMPLEMENTATION	40
6.1	Development Environment Setup	40
6.2	Tools and Technologies Used	40
6.3	Key Implementation of System Modules	40
6.3.1	AR Furniture Placement	41
6.3.2	Furniture Management	44
6.3.3	Cost Estimation	45
6.3.4	Cloud Persistence	48
6.3.5	Room Setup and Budget Initialization	52
7	UI DESIGN	55
7.1	Room Type Selection and Budget Setup Interface	55
7.2	Furniture Selection Panel	57
7.3	AR Design Workspace Interface	59
7.4	Saved Design Reload Interface	61

8	TESTING	63
8.1	Functional Testing	63
8.2	Performance Testing	63
8.3	Gesture-Based Interaction Testing	64
9	RESULTS AND DISCUSSION	65
10	GIT HISTORY	66
11	CONCLUSION	67
12	FUTURE WORK	68
13	APPENDIX	69
13.1	Minimum Software Requirements	69
13.2	Minimum Hardware Requirements	69
14	REFERENCES	70

1 INTRODUCTION

Interior design is a crucial part of constructing or renovating a home or office, as it directly influences comfort, functionality, and visual appeal. Making the right interior design decisions is often challenging because even small mistakes in furniture size, placement, or layout can negatively affect the overall feel of a space. People usually want to visualize how their rooms will look before finalizing designs, but this is not always easy during the planning stage. Traditionally, interior design relies on two dimensional drawings, blueprints, catalog images, and physical samples. Although these methods help in basic planning, they often fail to represent accurate scale, spatial relationships, and real world appearance. Users are required to imagine how furniture will fit into their rooms, which can lead to confusion, incorrect choices, and dissatisfaction once the design is implemented.

With the advancement of mobile technology, Augmented Reality (AR) has emerged as an effective solution to these challenges. Augmented Reality allows digital objects to be overlaid onto the real world environment in real time using a device camera. There are mainly two types of augmented reality techniques: marker-based AR, which requires predefined markers such as images or QR codes, and markerless AR, which detects real world surfaces like floors and walls without any physical markers. Markerless AR offers greater flexibility and usability, making it more suitable for interior design applications. The widespread availability of smartphones equipped with high resolution cameras, motion sensors, and tracking capabilities has made markerless AR accessible to everyday users. This technology enables users to place virtual objects naturally within their physical environment with accurate alignment, scaling, and interaction.

This project proposes a markerless AR based mobile application designed to simplify and enhance interior design visualization. The application allows users to select 3D furniture models and place them within their real environment using a smartphone. It also supports adjusting position, scale, and orientation of furniture models, along with saving interior design layouts through cloud integration for future reference. By providing realistic and interactive visualization, the proposed system helps users better understand spatial layouts, reduce design errors, and make more confident and informed interior design decisions.

2 SUPPORTING LITERATURE

2.1 Literature Review

Paper 1 : Cuevas JRG, et al. LayOut Loud: an AI powered augmented reality and mobile application for room interior design and layout optimization. In: Proceedings of the International Conference on Intelligent Cybernetics Technology Applications (ICICyTA) 2024; Bali, Indonesia. p. 888–894. doi:10.1109/ICICYTA64807.2024.10912928.

This research work presents a comprehensive approach to enhancing interior design through the integration of artificial intelligence (AI) and augmented reality (AR). The primary objective of this system is to improve user decision-making by providing personalized interior layout suggestions based on room dimensions, furniture types, and individual preferences.

The application leverages AI-driven algorithms to analyze user inputs and generate optimized layout recommendations, which are then visualized in real time using AR. A key contribution of this study lies in its use of markerless augmented reality, allowing users to visualize virtual furniture directly within their physical environment without relying on predefined markers. This significantly enhances usability and realism. The system also incorporates a 3D model viewer, enabling users to interact with furniture elements by rotating, repositioning, and previewing them within real spaces. Furthermore, the inclusion of clustering algorithms for furniture categorization helps streamline the selection process, reducing user effort and improving overall experience.

The authors evaluate the system using the ISO 25010:2023 software quality model, focusing on functional suitability, performance efficiency, and reliability. The evaluation results indicate a high level of user satisfaction, demonstrating the effectiveness of combining AI and AR for interior design applications. However, despite its strengths, the system introduces computational complexity due to the integration of AI based optimization, which may limit performance on low end devices. This study establishes that AI enhanced AR systems can significantly improve interior layout planning. At the same time, it highlights the need for balancing intelligent automation with system simplicity, which is an important consideration for practical AR-based interior design solutions.

Parameter	Description
Objective	To develop an AI-powered AR mobile application that generates personalized interior layouts
Technology Used	Artificial Intelligence, Augmented Reality, 3D Visualization
Methodology	Web-based AR system design and implementation
Algorithms / Techniques	Clustering algorithm for furniture categorization
System Workflow	User input → AI processing → 3D visualization → AR placement
Tools / Frameworks	Flutter, Firebase, Android Studio, Figma
Key Features	Personalized design suggestions, real-time AR visualization, furniture compatibility analysis
Evaluation Method	ISO 25010:2023 Quality Model with Likert Scale
Results	Improved accessibility and usability in AR furniture visualization
Advantages	Personalized recommendations, interactive AR experience, user-friendly design
Limitations	Dependent on device capability, accuracy of user input, learning curve

Table 2.1. *Reference paper1 summary*

Paper 2: TR S, Santhosh A, Ajith A, Gopi A. Interactive interior design using augmented reality and 3D modeling. In: Proceedings of the 2nd International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI); 2024; Coimbatore, India. p. 1056–1062. doi:10.1109/ICoICI62503.2024.10696390.

This paper investigates the application of augmented reality (AR) as a practical solution for enhancing visualization and user interaction in the interior design process. The authors discuss how traditional interior design methods rely heavily on two-dimensional drawings, static images, and physical samples, which often fail to convey realistic spatial perception and scale. To address these shortcomings, AR is introduced as a technology capable of linking conceptual design ideas with real-time visualization in actual physical environments.

The proposed system employs markerless augmented reality techniques combined with 3D modeling tools to enable users to place and interact with virtual furniture within real spaces. By scanning the surrounding environment and integrating the data into a Unity-based development platform, the application supports accurate placement, scaling, and alignment of furniture models. This approach allows users to explore multiple design alternatives efficiently, reducing dependency on imagination and minimizing the need for physical rearrangement of furniture.

The study highlights user experience as a key aspect of the system by incorporating intuitive touch-based interactions such as dragging, rotating, and resizing virtual objects. The authors also emphasize the importance of lighting conditions, rendering quality, and spatial accuracy in producing realistic and immersive AR experiences. However, the paper identifies certain limitations, including sensitivity to environmental lighting, dependency on device performance, and occasional inaccuracies in surface detection, which may affect consistency across different usage scenarios.

Overall, the research demonstrates that AR-based interior design systems significantly improve user engagement, spatial understanding, and design confidence compared to conventional techniques. The findings suggest that such interactive systems can support better communication between designers and users while reducing design errors. The study further indicates that continued improvements in tracking accuracy and device compatibility can enhance the practicality and scalability of AR-based interior design applications.

Parameter	Description
Objective	To enhance interior design visualization using AR and interactive 3D modeling
Technology Used	Augmented Reality, Unity 3D, VISLAM, 3D Modeling
Methodology	Step-based process: Scanning → Modeling → Alignment → Integration → Budget Estimation
Algorithms / Techniques	VISLAM (Visual-Inertial SLAM), SLAM
System Workflow	Scan space → Create/Load models → Align → Render in AR → Estimate cost
Tools / Frameworks	Unity 3D, Blender, Vuforia Engine
Key Features	Real-time visualization, environment scanning, budget estimation
Evaluation Method	System performance and functional validation
Results	Improved user interaction and design accuracy
Advantages	High realism, immersive experience, cost estimation support
Limitations	Requires high-end devices, sensitive to lighting/environment, limited accessibility

Table 2.2. *Reference paper 2 summary*

Paper 3: R NR, M R, S RB, Sultana S, Nadig NM. Markerless augmented reality application for interior designing. In: *Proceedings of the 2nd International Conference on Advanced Technologies in Intelligent Control, Environment, Computing & Communication Engineering (ICATIECE)*; 2022; Bangalore, India. p. 1–5. doi:10.1109/ICATIECE56365.2022.10047281.

This paper focuses on the use of markerless augmented reality for interior visualization and furniture planning. The study highlights the challenges faced by users when purchasing or arranging furniture without being able to visualize how items will appear within their actual living spaces. Traditional methods often require physical movement of furniture or reliance on imagination, which can lead to incorrect decisions and increased effort. To overcome these issues, the authors propose a markerless AR solution that allows users to place and examine virtual furniture models directly within their real environment using a mobile device.

The proposed system eliminates the dependency on physical markers and instead relies on camera input and sensor data to detect surfaces and determine spatial orientation. This approach improves usability and flexibility, enabling users to experiment with multiple furniture layouts quickly and efficiently. The application supports basic interactions such as object placement, movement, rotation, and repositioning, allowing users to gain a realistic preview of furniture arrangements before making final decisions. The study also explores web-based AR techniques to reduce application size and increase accessibility across different platforms.

One of the major advantages of the system is its emphasis on accessibility and convenience, as users can visualize furniture without visiting physical stores or installing heavy applications. The research highlights how markerless AR enhances user confidence during interior planning by providing real-time feedback and improved spatial understanding. However, the authors also identify limitations such as tracking instability under low-light conditions, performance variations across devices, and reduced accuracy in complex environments. These challenges indicate the need for further optimization in tracking algorithms and rendering techniques.

Overall, the study demonstrates that markerless augmented reality offers a practical and user-friendly approach to interior design visualization. The findings emphasize the potential of AR-based systems to improve decision-making, reduce design errors, and enhance user experience. The paper also suggests that future improvements in device capabilities and AR frameworks can further strengthen the reliability and realism of markerless AR applications for interior design.

Parameter	Description
Objective	To develop a web-based AR system for visualizing furniture without markers
Technology Used	WebAR, WebXR API, 3D Rendering
Methodology	Web-based AR system design and implementation
Algorithms / Techniques	Markerless tracking using WebXR
System Workflow	Scan environment → Detect surface → Place 3D model → Adjust → Confirm
Tools / Frameworks	React.js, Node.js, Express.js, MongoDB, Firebase, Three.js
Key Features	Markerless AR, browser-based access, real-time furniture placement
Evaluation Method	Functional implementation and performance observation
Results	Improved accessibility and usability in AR furniture visualization
Advantages	No app installation required, lightweight, cross-platform compatibility
Limitations	Limited placement control, dependency on browser/device support, tracking issues

Table 2.3. *Reference paper 3 summary*

2.2 Literature Review Summary

The reviewed literature highlights the growing importance of augmented reality (AR) as an effective tool for enhancing visualization and interaction in the interior design domain. The studies collectively demonstrate that traditional interior design approaches, which rely on two dimensional drawings, static images, and physical samples, often fail to provide accurate spatial perception and realistic understanding of furniture placement within real environments. The first paper emphasizes the integration of advanced technologies such as AI and AR to provide personalized and optimized interior layouts, showing that intelligent systems can significantly improve user decision-making and satisfaction. The second paper focuses on interactive AR-based interior design using 3D modeling, highlighting the role of markerless AR and intuitive user interactions in improving engagement, collaboration, and design clarity. The third paper concentrates on markerless AR applications for interior

designing, stressing accessibility, ease of use, and real-time visualization without the need for physical markers or complex setups. Across all three studies, markerless AR emerges as a preferred approach due to its flexibility and improved user experience. However, common challenges are also identified, including sensitivity to lighting conditions, dependency on device capabilities, surface detection inaccuracies, and performance variability across different environments. These limitations emphasize the importance of robust tracking, efficient rendering, and optimized interaction techniques. In addition, the literature indicates that user-centric design plays a crucial role in the success of AR-based interior design systems. Studies show that intuitive interfaces, smooth gesture controls, and realistic visualization significantly influence user acceptance and confidence.

Paper	Focus Area	Techniques Used	Key contributions
LayOut Loud – AI-Powered AR Interior Design (2024)	AI-assisted interior layout optimization with AR visualization	Markerless AR, AI-based layout optimization, clustering algorithms, 3D model visualization	Provides personalized interior layout suggestions; integrates AI with AR for optimized furniture placement; supports real-time visualization and interaction; evaluated using ISO 25010 quality standards
Interactive Interior Design Using AR and 3D Modeling (2024)	Interactive AR-based interior design visualization	Markerless AR, 3D modeling, Unity-based development	Enables accurate placement, scaling, and alignment of furniture; improves user engagement and collaboration; enhances spatial understanding through interactive visualization
Markerless Augmented Reality Application for Interior Designing (2022)	Simplified interior visualization using markerless AR	Markerless AR, camera and sensor-based tracking, mobile AR	Eliminates need for physical markers; improves accessibility and ease of use; enables real-time furniture placement and preview; increases user confidence in planning

Table 2.4. Literature review summary

2.3 Findings and Proposal

Interior design planning is inherently complex, as it requires an accurate understanding of spatial relationships, object dimensions, and layout arrangements. Traditional approaches such as two-dimensional drawings, static images, and physical samples are limited in their ability to represent real-world environments. These methods fail to provide depth perception and realistic context, making it difficult for users to accurately visualize how furniture and design elements will appear in an actual space. Consequently, users often make incorrect design decisions, leading to inefficient space utilization and reduced satisfaction.

Existing interior design solutions, including digital tools and catalog-based applications, largely rely on static visualization techniques. Although these systems provide convenience, they lack real-time interaction and do not offer accurate spatial mapping. Users are required to mentally interpret scale and positioning, which increases cognitive effort and leads to frequent design errors. As a result, user confidence in decision-making remains limited.

The literature review identifies a significant gap in the availability of lightweight, interactive, and user-friendly systems that provide real-time visualization without requiring complex setup or high computational resources. While some advanced systems incorporate artificial intelligence and high-end rendering techniques, they often suffer from performance limitations, increased complexity, and reduced accessibility on standard mobile devices. This highlights the need for a practical solution that balances performance, usability, and accuracy.

Furthermore, existing AR-based interior design systems face several technical challenges, including sensitivity to environmental conditions such as lighting, dependency on device hardware capabilities, inaccuracies in surface detection, and inconsistencies in tracking performance. Maintaining smooth interaction, spatial stability, and realistic visualization without introducing additional system overhead remains a critical challenge.

To address these limitations, this project proposes AR-Space: Virtual Interior Design using Augmented Reality, a markerless AR-based mobile application designed to provide real-time and interactive interior visualization. The system enables users to place, move, and rotate virtual furniture within their physical environment using a smartphone. Additionally, it incorporates a cloud-based persistence mechanism that allows users to save and reload design layouts, ensuring continuity and flexibility.

The proposed system emphasizes simplicity, spatial accuracy, and user-centric design. By integrating real-time AR interaction with cost awareness and persistent storage, the application aims to minimize design errors, enhance user decision-making, improve confidence, and provide a scalable foundation for future enhancements.

3 SYSTEM ANALYSIS

The AR-Space: Virtual Interior Design using Augmented Reality system is designed as a modular, markerless augmented reality application that enables users to visualize and arrange virtual furniture within their real-world environment.

The system utilizes mobile device sensors and camera input to detect real-world surfaces and accurately place virtual objects in real time. To ensure scalability, maintainability, and efficient performance, the system is divided into multiple functional modules, each responsible for a specific aspect of the application workflow.

3.1 Analysis of Existing Systems

Traditional interior design systems primarily rely on two-dimensional drawings, printed catalogs, and desktop-based computer-aided design (CAD) tools. These methods provide a basic representation of layouts but fail to convey accurate depth, scale, and spatial relationships. As a result, users must mentally visualize how furniture will appear in a real environment, which often leads to incorrect decisions and inefficient space utilization.

With the advancement of technology, digital tools such as 3D modeling software and mobile-based interior design applications have been introduced. These systems improve visualization by allowing users to view designs in a three-dimensional format. However, they still operate within a virtual environment and are not integrated with the user's physical surroundings. This limits their practical usefulness, as users cannot accurately assess how designs will look in real-world spaces.

Some modern solutions incorporate augmented reality (AR) to provide more immersive visualization. While these systems offer real-time placement of virtual objects, many suffer from issues such as unstable tracking, inaccurate plane detection, and lack of persistence. Objects may shift, misalign, or fail to maintain their position when the application is restarted, reducing reliability.

Additionally, many existing systems do not include essential decision-support features such as cost estimation and budget management. Users are required to depend on external tools to calculate expenses, which makes the overall process less efficient and less user-friendly.

Therefore, despite advancements in digital and AR-based interior design tools, there remains a gap in systems that provide accurate real-world visualization, stable object placement, integrated cost management, and persistent data storage within a single, user-friendly platform.

3.2 Limitations of Standard AR Placement

Standard augmented reality (AR) placement systems, although effective in providing basic real-time visualization, exhibit several limitations that affect their reliability and usability in practical interior design applications. One of the primary issues is instability in object placement caused by inaccuracies in plane detection and tracking. Virtual objects may appear to shift, jitter, or drift when the user moves the device, leading to an inconsistent and unrealistic experience.

A common problem observed in many AR systems is the orientation inconsistency issue, often referred to as the “180-degree flip bug.” In such cases, objects may appear incorrectly rotated or mirrored relative to the user’s viewpoint when reloading or repositioning, resulting in misaligned layouts. This occurs because many systems rely on absolute world coordinates without considering the user’s perspective at the time of placement.

Another significant limitation is the lack of persistent spatial mapping. In standard AR applications, once the session ends, the placed objects are lost, and there is no reliable mechanism to restore them in their original positions. Even when saving is implemented, the reloaded objects may not align correctly with the real-world environment due to changes in tracking or coordinate inconsistencies.

Additionally, many AR systems do not provide advanced interaction capabilities or integrated system features. Users often face limited control over object manipulation, and features such as cost estimation, budget validation, and layout management are typically absent. This reduces the practicality of such systems for real-world decision-making.

Environmental dependency is another challenge. AR performance is highly influenced by lighting conditions, surface textures, and device capabilities. Poor lighting or insufficient surface features can lead to incorrect plane detection and unstable object placement.

Therefore, while standard AR placement systems provide a foundation for interactive visualization, they lack the stability, persistence, and contextual accuracy required for reliable interior design applications. These limitations highlight the need for an improved approach that ensures consistent spatial alignment, robust data persistence, and enhanced user interaction.

3.3 Proposed Solution: User-Relative Coordinate Mapping

To overcome the limitations of conventional AR placement systems, the proposed solution introduces a user-relative coordinate mapping approach for accurate and consistent spatial reconstruction. Instead of storing furniture positions using absolute world coordinates, the system records object transformations relative to the user’s camera position and orientation at the time of placement.

In traditional AR systems, objects are saved based on global coordinate space, which can lead to misalignment when the session is restarted due to variations in tracking or environmental conditions. In contrast, the proposed approach captures the relative position and rotation of each object with respect to the user's viewpoint. This ensures that when a design is reloaded, the layout is reconstructed accurately from the user's current perspective.

The system calculates relative positions using inverse transformation techniques, where object positions are converted into the camera's local coordinate space. Similarly, object rotations are adjusted based on the camera's orientation, ensuring consistent alignment regardless of the device's movement or orientation changes.

Additionally, the solution incorporates a group-based layout reconstruction mechanism, where all furniture elements are instantiated as part of a unified layout group. This allows the entire design to be repositioned and aligned as a single entity before finalizing placement. Once aligned, the layout can be "locked," restoring individual interaction capabilities for each object.

To further enhance spatial accuracy, the system integrates floor snapping and surface validation techniques, ensuring that the reconstructed layout aligns correctly with detected real-world surfaces. This minimizes floating or misaligned objects and improves realism.

By combining user-relative coordinate mapping, grouped layout alignment, and surface-aware placement, the proposed system effectively eliminates issues such as spatial drift, orientation errors, and inconsistent object positioning. This results in a stable, reliable, and user-friendly AR interior design experience.

3.4 Project Pipeline

The project pipeline represents the sequence of steps involved in building and executing the system.

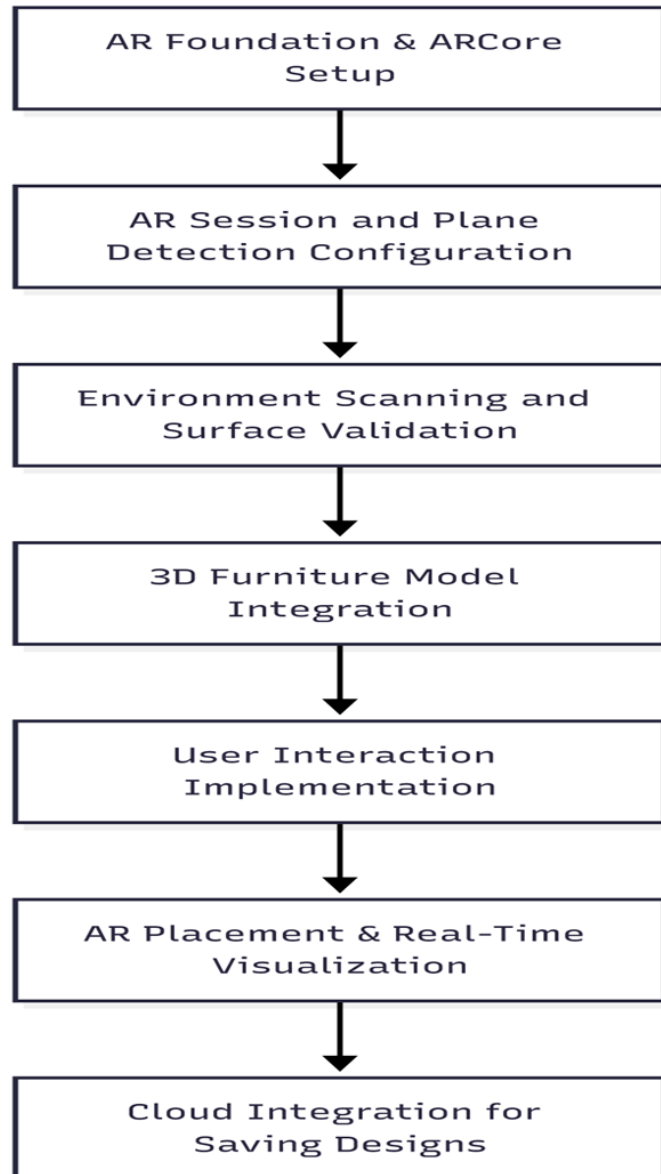


Figure 3.1. *Project Pipeline*

3.4.1 AR Foundation & ARCore Setup

This is the initial stage of the project pipeline, where the augmented reality environment is established. AR Foundation is integrated into the Unity development platform along with ARCore to enable markerless augmented reality capabilities. This setup provides the foundational support for motion tracking, camera access, and real-time interaction between virtual objects and the physical environment.

3.4.2 AR Session and Plane Detection Configuration

In this stage, the AR session is configured to manage the lifecycle of AR tracking. Plane detection features are enabled to identify real-world surfaces such as floors and walls. These detected planes act as reference points for placing virtual furniture models accurately within the physical space.

3.4.3 Environment Scanning and Surface Validation

After configuring plane detection, the system continuously scans the surrounding environment using the device camera and sensors. Detected surfaces are validated to ensure stability and suitability for object placement. This step improves accuracy and prevents incorrect or unstable placement of virtual furniture.

3.4.4 3D Furniture Model Integration

This stage involves integrating pre-built and optimized 3D furniture models into the application. The models are imported into Unity and prepared for real-time rendering. Proper integration ensures that furniture models maintain realistic appearance while minimizing performance overhead on mobile devices.

3.4.5 User Interaction Implementation

In this phase, user interaction mechanisms are implemented to allow intuitive control over virtual furniture. Touch-based gestures such as selection, movement, rotation, and scaling are enabled, allowing users to interact naturally with the furniture models in the augmented reality environment.

3.4.6 AR Placement & Real-Time Visualization

This stage combines plane detection, 3D models, and user interactions to achieve real-time AR visualization. Furniture models are placed on validated surfaces and rendered in true scale, providing a realistic and immersive interior design experience. Continuous tracking ensures smooth visualization as the user moves the device.

3.4.7 Cloud Integration for Saving Designs

The final stage of the pipeline focuses on integrating cloud storage functionality. Interior design layouts, including furniture positions and configurations, are saved to the cloud. This allows users to retrieve, modify, and reuse their designs across multiple sessions, providing flexibility and data persistence.

3.5 Feasibility Analysis

This section evaluates the feasibility of the proposed system to determine whether it can be effectively developed and implemented. It examines the system from technical, economic, and operational perspectives, focusing on the technologies used, overall cost, and ease of use. The analysis ensures that the system is practical, cost-efficient, and suitable for real-world interior design applications

3.5.1 Technical Feasibility

The system is technically feasible as it uses reliable technologies such as Unity, AR Foundation, and ARCore. These tools support key AR features like plane detection, motion tracking, and real-time rendering. The application runs on ARCore-supported Android devices without requiring additional hardware. Optimized 3D models and efficient memory management ensure smooth performance, while strong tool support reduces development complexity and risks

3.5.2 Economical Feasibility

The project is cost-effective as it uses freely available tools suitable for academic development. No additional hardware or paid software is required. The application runs on existing smartphones, making it affordable and practical. This low-cost approach also allows easy future enhancements without major financial investment. The project minimizes financial risk by utilizing open-source libraries and cloud tiers that offer generous free usage for development. Furthermore, by reducing the need for physical interior samples or professional consultations, the system provides significant long-term savings for the end user

3.5.3 Operational Feasibility

The system is easy to use due to its simple and intuitive interface. Users can interact using basic touch gestures such as placing, moving, and rotating furniture. It integrates well into the interior design process by providing real-time visualization and feedback. Minimal training is required, ensuring smooth adoption and usability. The application's workflow is designed to mirror real-world design habits, allowing for natural integration into existing construction or renovation planning. By providing immediate visual feedback and cost awareness, it empowers users to make confident decisions without requiring specialized technical expertise.

3.6 System Environment

The system environment outlines the specific hardware specifications and software frameworks required to successfully develop and execute the AR-Space application

3.6.1 Software Environment

The software environment includes the platforms, frameworks, programming languages, and tools used to design and implement the AR-Space application. These components provide the necessary support for markerless augmented reality, real-time rendering, and user interaction.

- **Unity Game Engine:** Unity serves as the core development platform for the application. It is used to create scenes, manage assets, integrate AR features, and handle real-time rendering of 3D furniture models.
- **AR Foundation:** AR Foundation provides a unified interface for implementing markerless AR functionalities. It supports plane detection, tracking, and light estimation, allowing the application to interact effectively with the real-world environment.
- **ARCore:** ARCore enables advanced AR features such as motion tracking, environmental understanding, and surface detection. It allows the application to accurately position virtual furniture within the physical space.
- **C# Programming Language:** C# is used for scripting application logic, managing user input, controlling object behavior, and handling AR session management.
- **Android SDK:** The Android Software Development Kit supports building, testing, and deploying the application on Android devices. It also facilitates debugging and performance optimization.
- **Pre-built 3D Furniture Models:** The application uses pre-existing and optimized 3D furniture models obtained from available libraries or asset repositories. These models are directly integrated into Unity without custom 3D modeling, ensuring realistic visualization while maintaining performance efficiency.

3.6.2 Hardware Environment

The hardware environment defines the physical devices required to develop and operate the AR-Space application. These components ensure accurate environment scanning, stable tracking, and smooth rendering of augmented reality content.

- **Android Smartphone with ARCore Support:** A compatible smartphone is required to run the application. The device must include a high-quality camera, motion sensors (accelerometer and gyroscope), and sufficient processing power to support real-time AR operations.
- **Development System (PC/Laptop):** A computer system is required for coding, designing, testing, and debugging the application. It should have adequate processing power and memory to handle Unity and 3D assets.
- **Camera and Sensors:** The device camera captures the real-world environment, while sensors provide motion and orientation data necessary for accurate plane detection and tracking.
- **Internet Connectivity:** Internet access is required for downloading software tools, assets, and updates. It also supports optional cloud-based features such as saving and retrieving interior design layouts.

4 SYSTEM DESIGN

The system design section details the comprehensive architectural framework of the AR-Space application. It outlines the layered AR Foundation structure and maps out the modular C# components that drive the application's real-time functionality.

4.1 System Architecture

The overall system architecture of the AR-Space application is based on the AR Foundation framework, which provides a unified interface for interacting with platform-specific AR technologies.

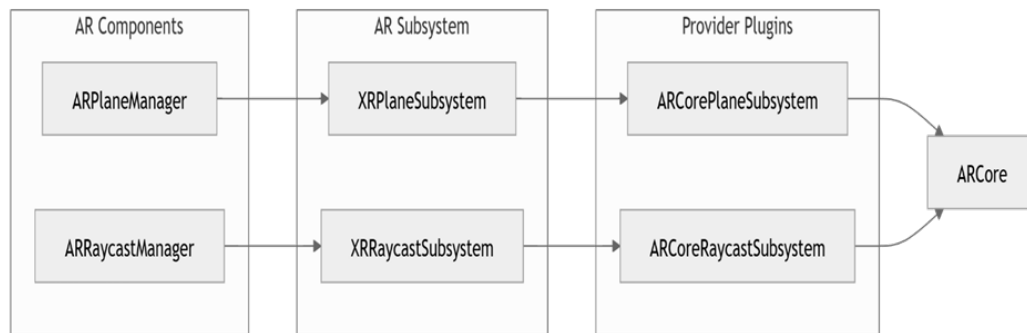


Figure 4.1. AR Foundation architecture diagram

The AR-Space application is built using Unity AR Foundation together with ARCore to enable markerless augmented reality functionality on Android devices. AR Foundation provides a unified framework that connects application-level AR components with platform-specific AR technologies. The architecture used in this project consists of three main layers: AR Components, AR Subsystems, and Provider Plugins, which together enable real-time interaction between virtual objects and the physical environment.

The AR Components layer represents the high-level components used directly by the application within the Unity environment. In this system, two primary components are used: ARPlaneManager and ARRaycastManager. The ARPlaneManager is responsible for detecting horizontal and vertical surfaces in the real-world environment using plane detection techniques. These detected planes serve as anchors where virtual furniture objects can be placed. The ARRaycastManager enables raycasting functionality, allowing the application to determine the exact point on a detected surface where a virtual object should be positioned based on the user's touch input on the device screen.

The second layer is the AR Subsystem layer, which acts as a platform-independent interface between the Unity components and the underlying AR platform. This layer ensures that AR functionality remains consistent regardless of the specific AR technology used. In this architecture, the ARPlaneManager communicates with the XRPlaneSubsystem, which

handles surface detection operations, while the `ARRaycastManager` interacts with the `XRRaycastSubsystem`, which manages raycasting operations used for object placement and interaction.

The third layer is the Provider Plugins layer, which contains the platform-specific implementations of the AR subsystems. In this project, the application uses `ARCore`, which provides the necessary functionality for motion tracking, environmental understanding, and surface detection on Android devices. The `ARCorePlaneSubsystem` processes plane detection data obtained from the device camera and sensors, while the `ARCoreRaycastSubsystem` handles raycasting operations used to determine accurate placement positions for virtual objects.

Finally, the `ARCore` engine interacts directly with the device hardware, including the camera and motion sensors such as the accelerometer and gyroscope. These sensors capture real-world spatial data and enable accurate tracking of the environment. This information is processed by `ARCore` and passed through the provider plugins and subsystems to the AR components in Unity, allowing the application to render and position virtual furniture models realistically within the user's physical space.

Through this layered architecture, AR Foundation simplifies the development of augmented reality applications by abstracting platform-specific details while maintaining efficient communication between the application, AR framework, and device hardware. This structure enables the AR-Space system to provide stable plane detection, precise object placement, and interactive visualization of interior designs in real time.

4.2 Core Functional Components

The AR-Space application follows a modular, manager-based architecture, where each core functionality is implemented using dedicated controller scripts. This design ensures clear separation of concerns, improves maintainability, and allows independent development and testing of each module. The modules communicate through singleton instances and shared data structures, enabling efficient coordination across the system.

4.2.1 ARFurniturePlacer – AR Placement Module

The AR Placement Module is the core component responsible for enabling real-time interaction between virtual furniture and the physical environment. It utilizes AR Foundation features such as `ARRaycastManager` to detect real-world surfaces and determine valid placement positions.

When the user interacts with the screen, the module performs raycasting operations to identify intersection points on detected planes. Based on this input, the selected furniture prefab is instantiated at the corresponding position and aligned with the surface. A slight

positional adjustment is applied to ensure proper grounding of objects.

In addition to placement, this module supports advanced interaction features such as object selection, dragging, rotation, and deletion. Drag operations update object positions dynamically based on continuous raycasting, while rotation is handled using gesture-based input. A double-tap mechanism is implemented for object deletion, ensuring intuitive control.

The module also integrates with the Furniture Manager and Cost Manager. Upon placement, objects are registered in the system and their associated cost is added. This tight integration ensures synchronization between AR interaction and system-level data management.

4.2.2 FurnitureManager – Furniture Management Module

The Furniture Manager Module acts as a centralized controller for handling all furniture-related operations within the system. It maintains a dynamic list of all furniture objects placed in the AR scene and provides methods to manage their lifecycle.

This module is responsible for storing the currently selected furniture prefab and making it accessible to the AR Placement Module during object instantiation. It also tracks all placed objects by adding them to an internal collection, ensuring that each object can be referenced for operations such as deletion, clearing, or saving.

Additionally, the module provides functionality to remove individual objects or clear all objects from the scene when required, such as during layout reset or when loading a new design. This centralized management ensures consistency across the application and simplifies interaction between different modules.

4.2.3 CostManager – Cost Estimation Module

The Cost Estimation Module is responsible for calculating, updating, and displaying the total cost of all furniture items placed in the AR environment. It maintains internal data structures to store item names, prices, and cumulative cost values.

When a furniture object is placed, the module retrieves its associated price either from a dedicated data component or through a pending price mechanism set during selection. The total cost is then updated dynamically, and the user interface is refreshed to reflect the new value.

The module also includes a budget validation system, where the total cost is continuously compared against a predefined or user-defined budget. If the total exceeds the budget, a visual warning is displayed, enhancing user awareness and decision-making.

Furthermore, the module generates an inventory summary that tracks the quantity of each item type. This summary is used during the cloud saving process.

4.2.4 CloudSaveManager – Cloud Save Module

The Cloud Save Module is responsible for enabling persistent storage of user designs using Unity Cloud Services. It handles authentication, data serialization, and communication with cloud storage systems.

When a user saves a design, the module packages layout data—including furniture positions, rotations, scale, cost information, and metadata—into a structured format. This data is then uploaded asynchronously to the cloud along with a compressed screenshot of the layout.

The module also supports retrieving all saved designs and loading specific layouts when requested. It ensures data integrity through unique identifiers (slot IDs), allowing each design to be uniquely tracked and managed.

Error handling mechanisms are included to manage network failures or service interruptions, ensuring reliability. This module enables users to store multiple layouts and access them across sessions, providing flexibility and continuity.

4.2.5 SaveandLoadUIManager – Save and Load UI Module

The Save and Load UI Module manages all user interactions related to saving, loading, and managing design layouts. It provides a dynamic interface for displaying saved designs as interactive slots with thumbnail previews and metadata.

When the user initiates a save operation, this module coordinates with the Screenshot Manager and Cloud Save Manager to capture the current layout and upload it to the cloud. It ensures that UI elements are temporarily hidden during screenshot capture to produce a clean image.

During loading, the module retrieves saved design data and dynamically generates UI cards for each slot. These cards include options for loading or deleting designs. The module also ensures proper synchronization between UI elements and backend data.

Additionally, it controls the visibility of save and reload buttons based on application state, ensuring that users can only perform actions when appropriate conditions are met.

4.2.6 RoomSetupManager – Room Setup Module

The Room Setup Module is responsible for configuring the initial state of the application based on user preferences. It provides a user interface for selecting room categories and entering budget values.

The module processes user input, validates budget values, and updates the Cost Manager accordingly. It also controls the visibility of various UI panels, ensuring that only relevant options are displayed based on the selected room type.

In addition, this module plays a critical role in loading saved layouts. It resets the current scene, restores budget settings, and reconstructs the layout using stored data. It also ensures that the AR environment is correctly initialized before placing objects.

This module acts as a bridge between user input and system initialization, ensuring a smooth transition from setup to interactive design.

4.2.7 LayoutAlignmentController – Layout Alignment Module

The Layout Alignment Module is designed to improve the usability of layout reconstruction after loading a saved design. When a layout is loaded, all furniture objects are grouped under a single parent container to maintain their relative spatial arrangement. This module allows users to reposition and align the entire layout as a single unit within the AR environment. This is particularly useful when the user's current position differs from the original placement context.

Once the layout is correctly aligned, the user can lock the layout, which removes the grouping and restores individual control over each object. This ensures both flexibility and precision in layout management. By combining grouping, alignment, and unlocking mechanisms, this module enhances spatial accuracy and provides a more intuitive user experience when working with saved designs.

4.3 AR Environment Setup

The AR Scene Setup defines the core components required to initialize and manage the augmented reality environment within the application. In Unity, this setup is implemented using AR Foundation, which provides a structured framework for handling real-world tracking, surface detection, and object placement. The AR scene consists of several essential components that work together to create a stable and interactive augmented reality experience.

4.3.1 AR Session

The AR Session is responsible for managing the lifecycle of the augmented reality experience. It initializes the AR system, starts tracking the environment, and maintains communication with the underlying AR platform (ARCore). When the application starts, the AR Session activates the camera feed and begins analyzing the real-world environment using device sensors. It continuously tracks the position and orientation of the device to ensure that virtual objects remain stable relative to the physical surroundings. Additionally, the AR Session handles events such as session reset, pause, and resume, ensuring smooth operation of the AR experience throughout the application usage.

4.3.2 AR Session Origin

The AR Session Origin acts as the reference coordinate system for all virtual objects in the AR scene. It defines the position and scale of the AR world relative to the real-world environment. All AR components such as the camera, detected planes, and placed objects are positioned relative to the AR Session Origin. When a user moves the device, the AR Session Origin updates the transformation of objects to maintain accurate alignment with the real-world space. This component ensures that virtual furniture appears fixed in the environment and does not drift or move incorrectly when the user changes their viewpoint.

4.3.3 AR Camera

The AR Camera captures the real-world environment and displays it as the background of the AR scene. It works together with ARCore to track device motion and orientation in real time. The camera continuously processes visual data and sensor inputs (such as accelerometer and gyroscope) to determine how the device is moving. This information is used to update the position of virtual objects so that they remain correctly aligned with the physical environment. The AR Camera also supports light estimation, which allows virtual objects to adjust their brightness and shading based on real-world lighting conditions, enhancing realism.

4.3.4 AR Plane Manager

The AR Plane Manager is responsible for detecting flat surfaces in the real-world environment, such as floors, tables, and walls. It uses computer vision techniques to analyze the camera feed and identify suitable surfaces for placing virtual objects. Once a surface is detected, the system creates a virtual plane that acts as an anchor point. These planes provide stable locations where furniture models can be placed accurately. The AR Plane Manager continuously updates detected surfaces as the user moves around, improving the accuracy and reliability of object placement. It also ensures that objects are placed only on valid and stable surfaces.

4.3.5 AR Raycast Manager

The AR Raycast Manager is used to determine the exact position where a virtual object should be placed in the AR environment. It performs a raycasting operation from the user's touch input on the screen into the real-world scene. When the user taps on the screen, a ray is projected from the camera into the detected planes. The system calculates the intersection point between the ray and the surface, and this point is used to place the virtual object. This mechanism enables precise and intuitive object placement, allowing users to interact naturally with the AR environment.

4.4 Application Workflow and User Interaction Diagram

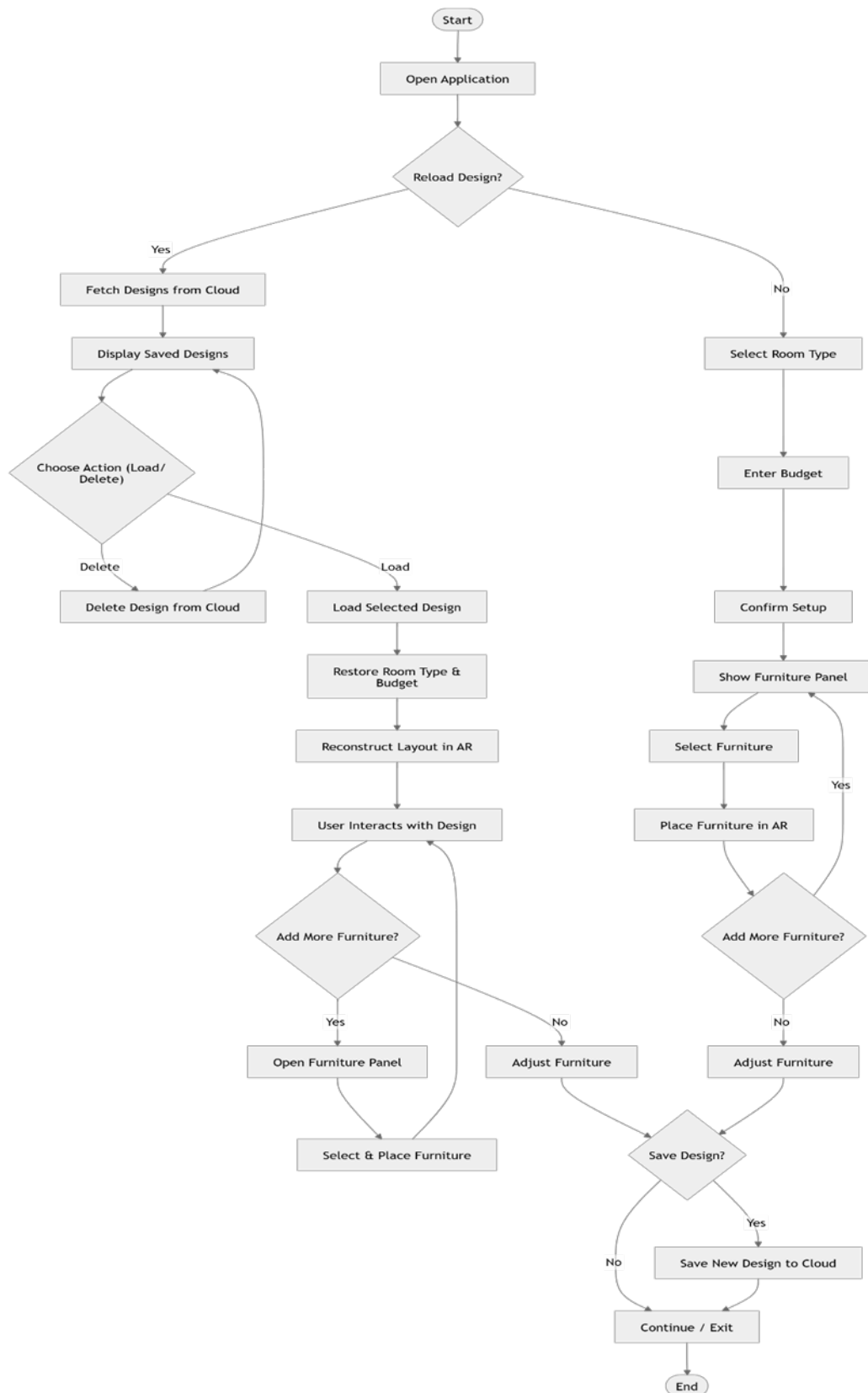


Figure 4.2. System workflow diagram

The Application Workflow and User Interaction Diagram illustrates the complete operational flow of the AR-based interior design application, covering both new design creation and existing design reloading processes.

When the application is launched, the user is first presented with an option to reload a previously saved design. If the user chooses to reload, the system retrieves all saved layouts from the cloud database and displays them in a gallery format. Each saved design is represented with a preview image and associated details.

At this stage, the user can choose between two actions: loading or deleting a design. If the user selects the delete option, the selected design is permanently removed from the cloud storage, and the updated list of saved designs is displayed again. If the user selects the load option, the system retrieves the corresponding layout data, restores the previously selected room type and budget, and reconstructs the furniture arrangement within the AR environment using stored spatial data. This allows the user to continue working on an existing design seamlessly.

Once a design is loaded, the user can interact with the placed furniture by moving, rotating, or adjusting objects within the AR scene. The user also has the option to add more furniture by opening the furniture panel, selecting additional items, and placing them in the environment. This process can be repeated multiple times until the desired layout is achieved.

If the user chooses not to reload a design, the application proceeds with the creation of a new layout. The user is prompted to select a room type and enter a budget. Upon confirming the setup, the system displays a furniture recommendation panel based on the selected room category. The user can then select furniture items and place them within the AR environment. After placing an item, the user can continue adding more furniture or proceed to adjust the existing arrangement.

Throughout both workflows, the user is provided with options to modify the layout dynamically, ensuring flexibility in design. Once the user is satisfied with the arrangement, they are prompted to save the design. If the save option is selected, the system captures the current layout, generates the necessary data representation, and stores it in the cloud as a new design entry. If the user chooses not to save, they can either continue editing or exit the application. The workflow concludes when the user finalizes the design or exits the application. Overall, the system supports seamless switching between new and existing designs, enabling users to manage, modify, and persist their interior layouts efficiently using augmented reality.

5 METHODOLOGY

The methodology of the proposed system describes the step-by-step process through which the AR-Space application achieves real-time interior visualization, user interaction, and data persistence. The system integrates augmented reality techniques with modular software design to enable accurate placement, manipulation, and storage of virtual furniture within a real-world environment. The methodology focuses on key processes such as AR-based placement, coordinate transformation, cost management, cloud-based storage, and layout reconstruction.

5.1 AR Placement Methodology

The AR placement methodology in the proposed system is based on markerless augmented reality, implemented using AR Foundation. This approach enables the application to detect real-world surfaces and place virtual furniture objects without requiring predefined markers.

Initially, the system continuously scans the surrounding environment using the device camera. The AR Plane Manager analyzes visual features such as edges, textures, and depth cues to identify flat surfaces like floors, walls, and tables. These detected surfaces are represented as trackable planes within the AR environment and serve as anchors for object placement.

When the user interacts with the screen (typically through a tap gesture), the system performs a raycasting operation using the AR Raycast Manager. A virtual ray is projected from the touch position on the screen into the real-world environment. If this ray intersects with a detected plane, a valid pose is obtained, consisting of position and orientation data.

Once a valid surface is identified, the selected furniture object (prefab) is instantiated at the detected position. The object is aligned with the surface orientation to ensure realistic placement. Additionally, a small vertical offset is applied to prevent the object from clipping into the surface, ensuring proper visual grounding.

After placement, the object becomes part of the interactive AR scene. The system enables further manipulation through gesture-based controls, allowing users to move, rotate, or delete objects as needed. Movement is achieved by continuously updating the object's position using real-time raycasting, while rotation is handled through gesture-based angle calculations.

To maintain system consistency, each placed object is registered within the Furniture Management Module, and its associated cost is updated through the Cost Estimation Module. This ensures synchronization between visual interaction and backend data processing.

The use of raycasting combined with plane detection provides a natural and intuitive “touch-to-place” interaction model. This methodology ensures accurate object placement, stable tracking, and seamless integration between the virtual and physical environments.

5.2 Camera-Relative Coordinate Translation

One of the key challenges in augmented reality applications is maintaining spatial consistency when reconstructing previously saved layouts. In conventional AR systems, object positions are stored using absolute world coordinates, which are highly dependent on the AR session's tracking state. When the application is restarted or the user's position changes, these coordinates often become unreliable.

To overcome this limitation, the proposed system introduces a camera-relative coordinate translation algorithm, which stores object transformations relative to the user's viewpoint instead of the global coordinate system.

Conceptual Approach

At the time of object placement, the system captures the current transformation of the device camera, including its position and orientation. Instead of directly storing the object's world position, the system computes its relative position with respect to the camera.

This is achieved using inverse transformation operations, where the object's position is converted into the camera's local coordinate space. Similarly, the object's rotation is adjusted relative to the camera's forward direction (heading). This ensures that the spatial relationship between the user and the object is preserved.

Mathematical Representation

Let:

- R_w = World position of the object
- C = Camera transform
- P_r = Relative position

The relative position is computed as:

$$P_r = C^{-1} \times R_w$$

Similarly, for rotation:

$$R_r = C^{-1} \times R_w$$

Where:

- C^{-1} represents the inverse transformation of the camera

- R_w is the world rotation of the object
- R_r is the relative rotation

These transformations ensure that all objects are stored in a coordinate system that is independent of the global AR session.

Layout Reconstruction Process

When a saved design is reloaded, the system performs the reverse transformation. The stored relative position is converted back into world coordinates using the current camera transform:

$$R_w = C \times P_r$$

$$R_w = C \times R_r$$

This process allows the layout to be reconstructed accurately relative to the user's current viewpoint, regardless of changes in device orientation or environment.

Handling Orientation Issues

A common issue in AR systems is the 180-degree flip problem where objects appear incorrectly rotated upon reloading. To address this, the system extracts only the horizontal orientation (yaw) of the camera and applies it during rotation reconstruction. This prevents unwanted tilting or flipping caused by variations in camera pitch and roll.

Integration with System Modules

This algorithm is tightly integrated with the Cloud Save Module, where relative coordinates are stored as part of the layout data, and the Room Setup Module, which handles reconstruction during loading. It also works in conjunction with the Layout Alignment Module, allowing users to fine-tune positioning after reconstruction.

5.3 Budget Management and Conditional Validation Logic

The proposed system incorporates a real-time budget management and validation mechanism to assist users in making cost-effective interior design decisions. This feature ensures that users remain aware of their spending while interacting with virtual furniture within the AR environment.

Cost Association and Initialization

Each furniture item available in the recommendation panel is associated with predefined metadata, including its name and price. When the user selects a furniture item, its corresponding price is extracted and temporarily stored as a pending value. This ensures that when the object is placed in the AR scene, the correct cost is assigned without requiring additional input.

Once the object is instantiated in the environment, the system retrieves its associated price and adds it to the cumulative total cost. This process is handled dynamically through the Cost Management module, ensuring that every placement action is immediately reflected in the total cost.

Dynamic Cost Calculation

The system maintains a running total of all placed furniture items. Internally, it uses structured data collections to store item names and corresponding prices. Whenever a new object is added, the system performs the following steps:

1. Retrieves the price of the selected furniture item
2. Adds the value to the total cost
3. Updates the user interface to display the new total

Similarly, when a furniture object is removed (for example, through a delete action), the system deducts its corresponding price from the total cost and updates the display accordingly. This ensures that the cost representation always reflects the current state of the design.

Budget Input and Validation

At the initial stage of the application, the user is required to enter a budget value during the room setup process. This value is parsed, validated, and stored within the system as the maximum allowable budget.

During interaction, the system continuously compares the total cost of placed items with the user-defined budget. This comparison forms the basis of the conditional validation logic.

Conditional Warning Mechanism

When the total cost exceeds the specified budget, the system triggers a visual warning. This is typically displayed as a highlighted message in the user interface, indicating that the user has exceeded their budget limit.

The warning mechanism is non-restrictive, meaning that users are still allowed to continue placing furniture even after exceeding the budget. This design choice ensures flexibility while still providing guidance.

$$\text{If Total Cost} > \text{Budget} \rightarrow \text{Display Warning}$$

This simple yet effective condition ensures real-time feedback without interrupting user interaction.

Inventory Summary Generation

In addition to total cost calculation, the system maintains an inventory summary, which tracks the number of instances of each furniture type placed in the scene. This is implemented using a mapping structure where each item name is associated with its count.

5.4 Cloud Persistence Method: Data Storage and Retrieval

The proposed system incorporates a cloud-based persistence mechanism to enable storage, retrieval, and management of interior design layouts across sessions. This ensures that users can save their work, revisit previous designs, and continue editing without losing data. The cloud persistence system is implemented using Unity Cloud Services and follows a structured approach for data handling, synchronization, and retrieval.

Data Collection and Serialization

When the user initiates a save operation, the system collects all relevant information required to reconstruct the design. This includes:

- Furniture object positions and rotations
- Relative coordinate data (camera-based transformation)
- Room type and category
- User-defined budget
- Total cost and inventory summary

Each furniture item is represented as a structured data object containing its transformation and identity details. These individual objects are aggregated into a larger SaveData structure, which represents the complete layout.

Before uploading, this data is converted into a serializable format (such as JSON-compatible structures), allowing it to be efficiently transmitted and stored in the cloud.

Unique Identification and Slot Management

To support multiple saved designs, each layout is associated with a unique identifier (slot ID). This identifier acts as a key for storing and retrieving data from the cloud.

The system maintains multiple save slots, where each slot represents a separate design configuration. This allows users to manage multiple interior layouts independently. Slot-based organization ensures that data remains structured and avoids overwriting existing designs unintentionally.

Asynchronous Cloud Storage

The system uses asynchronous operations to communicate with the cloud, ensuring that the application remains responsive during save and load operations. When a save request is triggered:

1. The serialized layout data is sent to the cloud database
2. A screenshot image is uploaded as a preview file
3. Both data and image are linked using the same slot identifier

Asynchronous handling prevents UI freezing and improves user experience, especially in mobile environments where network conditions may vary.

Screenshot Integration and Metadata Storage

In addition to layout data, the system captures a screenshot of the current AR scene during the save process. Before capturing, UI elements are temporarily hidden to ensure a clean and unobstructed image.

The screenshot is compressed and stored in the cloud as a preview thumbnail. This thumbnail is later displayed in the reload interface, allowing users to visually identify saved designs. Along with the image, metadata such as design name, timestamp, and inventory summary may also be stored to enhance usability.

Data Retrieval and Reconstruction

When the user chooses to reload a design, the system sends a request to the cloud to retrieve all stored layouts. The retrieved data includes both structured layout information and associated preview images.

Upon selecting a specific design:

1. The corresponding layout data is downloaded

2. The current scene is cleared to avoid conflicts
3. Furniture objects are reconstructed using stored relative coordinates
4. Room type, budget, and cost values are restored

This process ensures that the design appears exactly as it was saved, maintaining spatial consistency and user context.

Data Integrity and Reliability

To ensure reliable operation, the system incorporates basic error handling mechanisms. These include:

- Handling network failures during upload/download
- Preventing incomplete data writes
- Ensuring synchronization between layout data and screenshot files

The use of unique identifiers ensures that each design is independently managed, reducing the risk of data corruption or overwriting.

Integration with System Modules

The cloud persistence mechanism interacts with multiple modules:

- **Cost Manager:** Provides total cost and inventory data
- **Room Setup Module:** Restores room configuration and budget
- **Layout Reconstruction Module:** Rebuilds furniture arrangement
- **UI Module:** Displays saved designs and manages user interaction

This integration ensures a seamless workflow between saving, loading, and editing operations.

Advantages of Cloud-Based Persistence

- Enables cross-session continuity of designs
- Supports multiple layout management
- Provides visual preview using screenshots
- Enhances user flexibility and convenience
- Ensures data safety and accessibility

5.5 Screenshot Capture, Buffer Management, and UI Handling

The proposed system incorporates a screenshot capture mechanism to generate visual previews of interior design layouts during the save process. This feature enhances usability by allowing users to easily identify and manage their saved designs through thumbnail images.

Screenshot Capture Process

When the user initiates a save operation, the system captures the current AR scene as an image. This is achieved by reading pixel data directly from the screen buffer using Unity's rendering pipeline.

The capture process is performed at the end of the rendering frame to ensure that all visual elements, including placed furniture and environmental details, are fully rendered before the image is recorded. This guarantees that the screenshot accurately reflects the current state of the design.

UI Visibility Management

To ensure that the captured image is clean and free from interface elements, the system temporarily disables all UI components before taking the screenshot. This includes buttons, panels, text overlays, and other interactive elements.

Once the screenshot is captured, the UI elements are restored to their original state. This process is handled seamlessly, ensuring that the user does not experience any noticeable disruption.

This approach ensures that the saved image represents only the AR scene, improving clarity and professionalism in the preview thumbnails.

Image Processing and Compression

After capturing the raw screen image, the system processes it to optimize storage and transmission. Since full-resolution images can be large in size, the system generates a scaled-down thumbnail version of the screenshot.

The image is then compressed using a suitable encoding format (such as JPEG), balancing image quality and file size. This reduces storage requirements and ensures faster upload and retrieval from the cloud.

Buffer Management and Performance Considerations

Efficient buffer management is critical to maintaining application performance, especially on mobile devices. The system ensures that memory allocated for image processing is properly managed and released after use.

Temporary textures created during the capture process are cleared or reused to prevent memory leaks. Additionally, the screenshot operation is performed in a controlled manner to avoid frame drops or performance degradation during real-time AR interaction.

Integration with Cloud Storage

Once the screenshot is processed and compressed, it is uploaded to the cloud alongside the layout data. The image is associated with the same unique identifier (slot ID) used for the design, ensuring proper linkage between data and its visual representation.

During the reload process, these images are retrieved and displayed in the user interface as preview thumbnails, enabling quick and intuitive selection of saved designs.

User Experience Enhancement

The screenshot mechanism significantly improves user experience by:

- Providing visual identification of saved layouts
- Making the reload interface more intuitive
- Reducing confusion when managing multiple designs
- Enhancing the overall professionalism of the application

5.6 Layout Reload Methodology

The layout reload methodology is responsible for accurately restoring previously saved interior designs within the augmented reality environment. This process ensures that all furniture objects, along with their spatial relationships, are reconstructed reliably across different sessions.

Retrieval of Saved Data

When the user selects a saved design from the reload interface, the system initiates a cloud request to fetch the corresponding layout data. This data includes:

- Relative positions and rotations of furniture objects
- Furniture identifiers (prefab references)
- Room type and category
- Budget and cost information
- Inventory summary

The retrieved data is then parsed and prepared for reconstruction within the AR environment.

Scene Reset and Preparation

Before reconstructing the layout, the system clears all existing objects from the current scene. This step is essential to prevent overlap, duplication, or conflicts between previously placed objects and the new layout.

The AR environment remains active, ensuring that plane detection and tracking continue seamlessly. The system also restores the previously saved room configuration and budget settings to maintain consistency with the original design.

Object Reinstantiation

Each furniture item stored in the layout data is recreated by instantiating the corresponding prefab. The system uses the stored identifiers to match each item with its correct 3D model. Instead of directly applying world coordinates, the system uses camera-relative transformation data to determine the correct position and orientation of each object. This ensures that the layout remains consistent regardless of changes in the user's current position or AR session state.

Spatial Reconstruction Using Relative Coordinates

The reconstruction process involves converting stored relative coordinates back into world coordinates using the current camera transform. This is achieved through forward transformation operations, where each object's position and rotation are recalculated based on the camera's current orientation.

This approach ensures that:

- Objects maintain their original spatial relationships
- Layout orientation remains consistent
- Errors such as drift or misalignment are minimized

By avoiding dependence on global coordinate systems, the system achieves reliable and repeatable layout reconstruction.

Layout Grouping Mechanism

After all objects are instantiated, they are temporarily grouped under a single parent container. This grouping preserves the relative arrangement of all furniture elements and allows the entire layout to be treated as a single unit.

This mechanism is particularly useful when the user's current position differs from the original placement context, as it enables quick repositioning of the entire design.

Post-Load Interaction and Adjustment

Once the layout is reconstructed, the user is allowed to interact with the design. The grouped layout can be repositioned and aligned within the AR environment using alignment controls. After alignment, the layout can be "unlocked," which removes the grouping and restores individual interaction capabilities for each object. Users can then:

- Move individual furniture items
- Rotate objects
- Add new furniture
- Remove existing items

This ensures flexibility while maintaining the integrity of the original design.

Synchronization with System Modules

The layout reload process integrates with multiple modules:

- **Room Setup Module:** Restores room type and budget
- **Furniture Manager:** Tracks all instantiated objects
- **Cost Manager:** Recalculates and updates total cost
- **Layout Alignment Module:** Handles grouping and repositioning

This coordinated interaction ensures that all aspects of the system remain synchronized after loading a design.

5.7 Layout Alignment and Interaction Refinement

The Layout Alignment and Interaction Refinement methodology is designed to improve the usability and accuracy of spatial positioning after a layout is reconstructed in the AR environment. Since reloaded layouts are generated relative to the user's current viewpoint, there may be slight positional offsets depending on the user's physical location. This methodology ensures that the entire layout can be easily repositioned and refined before final interaction.

Need for Layout Alignment

Although the camera-relative coordinate system ensures correct relative positioning between objects, the overall layout may not always align perfectly with the real-world floor or desired placement area after loading. This can occur due to differences in user position, orientation, or environmental conditions at the time of reconstruction.

To address this, the system introduces a mechanism that allows users to adjust the entire layout as a single unit before interacting with individual objects.

Group-Based Alignment Approach

Once all furniture objects are reconstructed, they are temporarily grouped under a single parent container (layout group). This grouping preserves the relative arrangement of all objects and allows them to be manipulated collectively.

By treating the layout as a unified structure, the user can reposition the entire design within the AR environment without disturbing the internal arrangement of furniture items.

Alignment Controls and Interaction

The system provides intuitive controls that enable users to adjust the position and orientation of the grouped layout. These controls typically include:

- **Translation:** Moving the entire layout across the detected plane
- **Rotation:** Rotating the layout around a vertical axis for proper alignment
- **Floor Snapping:** Ensuring that all objects remain aligned with the detected surface

These interactions are implemented using gesture-based input or UI controls, allowing users to perform adjustments naturally within the AR scene.

Layout Locking and Ungrouping

After the user completes alignment, the system allows the layout to be “locked.” During this process:

1. The parent grouping is removed
2. Each object is detached from the layout container
3. Individual interaction capabilities are restored

This transition enables users to switch from global layout adjustment to fine-grained object manipulation.

Interaction Refinement

Once the layout is unlocked, users can refine their design by:

- Adjusting individual furniture positions
- Rotating specific objects
- Adding new items to the layout
- Removing unwanted objects

The system ensures that all interactions remain smooth and responsive, maintaining consistency with the AR placement methodology.

Integration with Other Modules

The layout alignment process is closely integrated with multiple system components:

- **Layout Reload Module:** Provides grouped objects for alignment
- **AR Placement Module:** Enables interaction after unlocking
- **Furniture Manager:** Maintains updated object references
- **Cost Manager:** Updates cost if objects are modified

This integration ensures that alignment operations do not disrupt system functionality or data consistency.

6 IMPLEMENTATION

This chapter details the practical development of the AR-Space application, outlining the environment setup, utilized technologies, and the core code implementations that drive the system's functionality.

6.1 Development Environment Setup

The AR-Space application was developed using Unity 2022.3, which provides a stable and feature-rich platform for building real-time 3D and augmented reality applications. The project utilizes AR Foundation as the primary framework for implementing markerless AR functionality, enabling compatibility with ARCore-supported Android devices.

The application is developed using the C# programming language, which is the standard scripting language in Unity. For cloud-based storage and data persistence, Unity Cloud Services (UGS) is used, providing secure and scalable backend support.

The development and testing were carried out on ARCore-supported Android devices to ensure proper functionality of plane detection, object placement, and user interaction.

6.2 Tools and Technologies Used

- **Unity Engine:** Core platform for development and rendering
- **AR Foundation:** Framework for AR interaction and plane detection
- **ARCore:** Provides device-level AR capabilities
- **C #:** Programming language for scripting
- **Unity Cloud Services:** Used for data storage and retrieval
- **TextMeshPro:** For advanced UI text rendering
- **Unity Gaming Services:** serverless backend

6.3 Key Implementation of System Modules

This section presents the implementation of the core system modules used in the application. Relevant portions of the code are included to highlight key functionalities such as AR object placement, interaction handling, cost management, cloud persistence, and layout reconstruction. Only essential code segments are provided for clarity and better understanding of the system workflow.

6.3.1 AR Furniture Placement

AR Furniture Placement using Raycasting

```

149     GameObject prefab = FurnitureManager.Instance.GetSelectedFurniture();
150
151     if (prefab == null)
152     {
153         Debug.Log("No furniture selected ✖");
154         return;
155     }
156
157     if (raycastManager.Raycast(touchPosition, hits, TrackableType.PlaneWithinPolygon))
158     {
159         Pose pose = hits[0].pose;
160
161         Debug.Log("Spawning: " + prefab.name);
162
163         GameObject obj = Instantiate(prefab, pose.position, pose.rotation);
164
165         // Slight lift
166         obj.transform.position += Vector3.up * 0.02f;
167
168         GameObject rootObj = obj.transform.root.gameObject;
169
170         Debug.Log("Registering object: " + rootObj.name);
171
172         // 1. Register the placed object
173         FurnitureManager.Instance.RegisterPlacedObject(rootObj);
174
175         // 2. Add cost for the placed object
176         if (CostManager.Instance != null)
177         {
178             CostManager.Instance.AddItem(rootObj);
179         }
180
181         // 3. Clear selection AFTER everything is done
182         FurnitureManager.Instance.ClearSelection();
183
184         Debug.Log("Furniture placed + cost updated ✔");
185     }
186     else
187     {
188         Debug.Log("No plane detected ✖");
189     }
190
191     lastTapTime = Time.time;
192 }

```

Figure 6.1. AR Furniture Placement using Raycasting

The code above performs surface detection using raycasting and places the selected furniture object at the detected position. The object is instantiated using the detected pose and slightly adjusted for proper alignment. It is then registered in the system, and its cost is added dynamically. Finally, the selection is cleared for further interactions.

Object Selection

```
// SELECT OBJECT
// =====
if (Physics.Raycast(ray, out hit))
{
    if (hit.transform.CompareTag("Furniture"))
    {
        selectedObject = hit.transform.root.gameObject;
        Debug.Log("Selected object 🖱️");
        lastTapTime = Time.time;
        return;
    }
}
```

Figure 6.2. *Object Selection*

The code above uses raycasting to detect user interaction with objects in the AR scene. When a furniture object is tapped, it is identified using its tag and assigned as the currently selected object. This enables further operations such as movement, rotation, or deletion on the selected item.

Drag and Reposition

```
// =====
#if UNITY_EDITOR
    if (Input.GetMouseButton(0) && selectedObject != null)
#else
    if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Moved && selectedObject != null)
#endif
    {
        if (raycastManager.Raycast(touchPosition, hits, TrackableType.PlaneWithinPolygon))
        {
            Pose pose = hits[0].pose;
            selectedObject.transform.position = pose.position;
        }
        return;
    }
```

Figure 6.3. *Drag and Reposition of Furniture Object*

This code enables movement of a selected furniture object based on user input. When the user drags on the screen, a raycast is performed to detect a valid surface, and the selected object's position is updated accordingly. This allows smooth and intuitive repositioning of furniture within the AR environment.

Rotation of Furniture

```
#if !UNITY_EDITOR
    if (Input.touchCount == 2 && selectedObject != null)
    {
        Touch t0 = Input.GetTouch(0);
        Touch t1 = Input.GetTouch(1);

        Vector2 prevDir = (t0.position - t0.deltaPosition) - (t1.position - t1.deltaPosition);
        Vector2 currDir = t0.position - t1.position;

        float angle = Vector2.SignedAngle(prevDir, currDir);

        selectedObject.transform.Rotate(Vector3.up, angle);

        return;
    }
#endif
```

Figure 6.4. *Rotation of Furniture using Multi-Touch Gesture*

The code above enables rotation of the selected furniture object using a two-finger touch gesture. It calculates the angle between previous and current touch positions and applies rotation around the vertical axis. This allows intuitive and precise orientation adjustment of objects in the AR environment.

Double-Tap Deletion

```
#if !UNITY_EDITOR
    if (Input.GetMouseButtonDown(0) && timeSinceLastTap <= doubleTapThreshold)
#else
    if (Input.touchCount > 0 && Input.GetTouch(0).phase == TouchPhase.Began && timeSinceLastTap <= doubleTapThreshold)
#endif
    {
        if (Physics.Raycast(ray, out hit))
        {
            if (hit.transform.CompareTag("Furniture"))
            {
                Debug.Log("Deleting object 🗑️");

                GameObject rootObj = hit.transform.root.gameObject;

                // 🔥 REMOVE FROM COST SYSTEM FIRST
                if (CostManager.Instance != null)
                {
                    CostManager.Instance.RemoveItem(rootObj);
                }

                // Existing delete system
                FurnitureManager.Instance.DeletePlacedObject(rootObj);
            }
        }

        lastTapTime = 0f;
        return;
    }
}
```

Figure 6.5. *Double-Tap Deletion of Furniture Object*

The code implements deletion of furniture objects using a double-tap gesture. When a valid object is detected through raycasting, it is removed from both the Cost Manager and Furniture Manager. This ensures that the object is deleted from the scene and its associated cost is updated accordingly.

6.3.2 Furniture Management

```
// =====  
// REGISTER PLACED OBJECT  
// =====  
public void RegisterPlacedObject(GameObject obj)  
{  
    if (obj == null)  
    {  
        Debug.LogError("❌ Placed object is NULL");  
        return;  
    }  
  
    placedObjects.Add(obj);  
  
    Debug.Log("✅ Object registered: " + obj.name);  
  
    // Selection is cleared by ARFurniturePlacer after placement is fully complete  
}
```

Figure 6.6. *Registering and Maintaining Placed Furniture Objects*

The code registers each placed furniture object into a centralized list for tracking. It ensures that only valid objects are added and maintains a record of all active objects in the scene. This enables efficient management of operations such as deletion, cost calculation, and layout reconstruction.

6.3.3 Cost Estimation

Dynamic Cost Addition

```
public void AddItem(GameObject obj)
{
    if (obj == null)
    {
        Debug.LogError("❌ AddItem received NULL object");
        return;
    }

    Debug.Log("🔥 AddItem CALLED for: " + obj.name);

    // Try to get FurnitureData from the object
    FurnitureData data = obj.GetComponentInChildren<FurnitureData>(true);

    if (data != null)
    {
        // Use FurnitureData if available
        itemNames.Add(data.furnitureName);
        itemPrices.Add(data.price);
        totalCost += data.price;

        Debug.Log("🔥 Added: " + data.furnitureName + " | Rs. " + data.price);
    }
    else if (pendingItemPrice > 0)
    {
        // Use pending price from FurnitureCardUI
        itemNames.Add(pendingItemName);
        itemPrices.Add(pendingItemPrice);
        totalCost += pendingItemPrice;

        Debug.Log("🔥 Added (from card): " + pendingItemName + " | Rs. " + pendingItemPrice);

        // Clear pending
        pendingItemName = "";
        pendingItemPrice = 0f;
    }
}
```

Figure 6.7. *Dynamic Cost Addition for Placed Furniture*

The code dynamically adds the cost of each placed furniture item to the total cost. It retrieves the price from the `FurnitureData` component and updates the total accordingly. If the data is unavailable, it uses a fallback value from the UI. This ensures accurate and real-time cost calculation.

Cost Removal

```

public void RemoveItem(GameObject obj)
{
    if (obj == null) return;

    // Try FurnitureData first
    FurnitureData data = obj.GetComponentInChildren<FurnitureData>(true);

    if (data != null && itemNames.Count > 0)
    {
        int idx = itemNames.IndexOf(data.furnitureName);
        if (idx >= 0)
        {
            totalCost -= itemPrices[idx];
            itemNames.RemoveAt(idx);
            itemPrices.RemoveAt(idx);
            Debug.Log("🗑️ Removed: " + data.furnitureName);
        }
    }
    else if (itemNames.Count > 0)
    {
        // Remove the last item if we can't identify it
        int lastIdx = itemNames.Count - 1;
        totalCost -= itemPrices[lastIdx];
        Debug.Log("🗑️ Removed: " + itemNames[lastIdx]);
        itemNames.RemoveAt(lastIdx);
        itemPrices.RemoveAt(lastIdx);
    }

    Debug.Log("💡 New Total: " + totalCost);
    UpdateUI();
}

```

Figure 6.8. *Cost Removal upon Deleting Furniture*

The above code handles the removal of cost when a furniture object is deleted from the scene. It identifies the corresponding item, subtracts its price from the total cost, and updates the tracking lists. This ensures that the total cost remains accurate and consistent after object deletion.

Budget Validation and Warning

```

public void UpdateUI()
{
    if (totalCostText != null)
    {
        totalCostText.enableAutoSizing = true;
        totalCostText.fontSizeMin = 14;
        totalCostText.fontSizeMax = 43;
        totalCostText.color = Color.white; // Force text explicitly to White

        // Also ensure the parent background is clearly colored
        if (totalCostText.transform.parent != null)
        {
            UnityEngine.UI.Image bgImage = totalCostText.transform.parent.GetComponent<UnityEngine.UI.Image>();
            if (bgImage != null)
            {
                if (maxBudget > 0 && totalCost > maxBudget)
                {
                    // Exceeded Budget - Red Warning
                    totalCostText.text = "Total Cost: Rs. " + totalCost.ToString("N0") + "\n<color=FF5555>▲ OVER BUDGET!</color>";
                    bgImage.color = new Color(0.8f, 0f, 0f, 0.8f); // Flashy semi-transparent Red
                }
                else
                {
                    // Under Budget - Normal
                    totalCostText.text = "Total Cost: Rs. " + totalCost.ToString("N0") + "\n<size=50%><color=AAAAAA>Budget: Rs ";
                    bgImage.color = new Color(0f, 0f, 0f, 0.6f); // Normal semi-transparent Black
                }
            }
        }
    }
}

```

Figure 6.9. Budget Validation and Warning Display

The code updates the user interface based on the total cost and budget. When the total cost exceeds the defined budget, a warning message is displayed along with a color change to indicate the over-budget condition. If the cost is within the budget, normal information is shown, ensuring clear and real-time feedback to the user.

6.3.4 Cloud Persistence

ID Generation and Layout Data Collection

The code below generates a unique identifier for each design using a timestamp and GUID. It then collects layout details such as furniture items, total cost, budget, and room category. The data is structured and prepared for storage, ensuring accurate reconstruction of the design during the reload process.

```
{
    // 🔒 TRANSACTIONAL LOCK: Generate Unique GUID and Freeze Room Data IMMEDIATELY
    string timeStamp = System.DateTime.Now.ToString("yyyyMMdd_HHmmss");
    string uniqueGuid = System.Guid.NewGuid().ToString().Substring(0, 8);
    string slotId = $"SaveSlot_{timeStamp}_{uniqueGuid}";

    // 3. Compile layout data (FROZEN SCAN - Before screenshot starts!)
    SaveData dataToSave = new SaveData();
    dataToSave.slotId = slotId; // Embed fingerprint into the JSON
    dataToSave.inventorySummary = new Dictionary<string, int>();
    float calculatedTotal = 0;
    foreach (GameObject obj in FurnitureManager.Instance.GetPlacedObjects())
    {
        FurnitureData fData = obj.GetComponentInChildren<FurnitureData>(true);
        string fName = fData != null ? fData.furnitureName : obj.name.Replace("(Clone)", "").Trim();
        float fPrice = fData != null ? fData.price : 0;

        calculatedTotal += fPrice;

        if (dataToSave.inventorySummary.ContainsKey(fName))
        {
            dataToSave.inventorySummary[fName]++;
        }
        else
        {
            dataToSave.inventorySummary[fName] = 1;
        }
    }
    dataToSave.totalCost = calculatedTotal;
    // 📍 Capture Context: Budget & Category
    if (CostManager.Instance != null)
    {
        dataToSave.savedBudget = CostManager.Instance.maxBudget;
    }
    RoomSetupManager setup = FindObjectOfType<RoomSetupManager>();
    if (setup != null && setup.roomDropdown != null)
    {
        dataToSave.roomCategory = setup.roomDropdown.options[setup.roomDropdown.value].text;
    }
    dataToSave.furnitureItems = new List<SavedFurnitureItem>();
    // Find reference point for spatial grouping
    Vector3 camPos = Camera.main.transform.position;
    float saveFloorY = camPos.y - 1.4f;
    ARRaycastManager raycastManager = FindObjectOfType<ARRaycastManager>();
    if (raycastManager != null)
    {
        List<ARRaycastHit> hits = new List<ARRaycastHit>();
        if (raycastManager.Raycast(new Ray(camPos, Vector3.down), hits, TrackableType.PlaneWithinPolygon))
        {
            saveFloorY = hits[0].pose.position.y;
        }
    }
}
```

Figure 6.10. *Unique ID Generation and Layout Data Collection*

Screenshot Capture and Processing

```
private IEnumerator CaptureRoutine(System.Action<byte[]> onCaptureComplete)
{
    // 1. Hide UI
    foreach (var ui in uiElementsToHide)
    {
        if (ui != null) ui.SetActive(false);
    }

    // 2. Wait for UI to actually hide and frame to finish rendering
    yield return new WaitForEndOfFrame();

    // 3. Take screenshot
    Texture2D screenImage = new Texture2D(Screen.width, Screen.height, TextureFormat.RGB24, false);
    screenImage.ReadPixels(new Rect(0, 0, Screen.width, Screen.height), 0, 0);
    screenImage.Apply();

    // 4. Resize to a thumbnail to save cloud space (e.g. 512x288 depending on aspect ratio)
    Texture2D thumbnail = ResizeTexture(screenImage, 512);

    // 5. Encode to JPG (highly compressed compared to PNG)
    byte[] imageBytes = thumbnail.EncodeToJPG(75);
}
```

Figure 6.11. *Screenshot Capture and Processing*

The above code captures the current AR scene by temporarily hiding UI elements and reading pixel data from the screen. The captured image is then resized and compressed into a smaller format before being stored. This ensures efficient storage and provides a clear visual preview of saved designs

Saving Layout Data

```
public async Task<bool> SaveDesignAsync(string slotId, SaveData payload, byte[] screenshotBytes)
{
    try
    {
        // 1. Save JSON Metadata
        var data = new Dictionary<string, object>
        {
            { slotId, payload }
        };
        await CloudSaveService.Instance.Data.Player.SaveAsync(data);
        Debug.Log($"Saved layout metadata to slot {slotId}");

        // 2. Save Screenshot File
        string filename = $"{slotId}_screenshot.png";
        await CloudSaveService.Instance.Files.Player.SaveAsync(filename, screenshotBytes);
        Debug.Log($"Saved screenshot file {filename}");

        return true;
    }
    catch (Exception e)
    {
        Debug.LogError($"Error saving design: {e.Message}");
        return false;
    }
}
```

Figure 6.12. *Saving Layout Data and Screenshot to Cloud*

The above code saves the design data to the cloud by storing both layout metadata and the corresponding screenshot. The layout information is saved as structured data, while the

screenshot is stored as an image file. This ensures that each design is preserved along with a visual preview for easy identification during retrieval.

Retrieving Saved Designs from Cloud

```
public async Task<Dictionary<string, SaveData>> GetAllSavedDesignsAsync()
{
    try
    {
        var results = await CloudSaveService.Instance.Data.Player.LoadAllAsync();
        Dictionary<string, SaveData> designs = new Dictionary<string, SaveData>();

        foreach (var item in results)
        {
            if (item.Key.StartsWith("SaveSlot_"))
            {
                SaveData data = item.Value.Value.GetAs<SaveData>();
                designs.Add(item.Key, data);
            }
        }
        return designs;
    }
    catch (Exception e)
    {
        Debug.LogError($"Error loading designs: {e.Message}");
        return new Dictionary<string, SaveData>();
    }
}
```

Figure 6.13. *Retrieving Saved Designs from Cloud*

The above code retrieves all saved design data from the cloud storage. It filters relevant entries based on slot identifiers and converts them into structured objects. The retrieved designs are stored in a dictionary, enabling efficient access and display in the user interface.

Deleting Saved Design from Cloud

```
public async Task<bool> DeleteDesignAsync(string slotId)
{
    try
    {
        // Delete metadata
        await CloudSaveService.Instance.Data.Player.DeleteAsync(slotId);

        // Delete screenshot file
        string filename = $"{slotId}_screenshot.png";
        await CloudSaveService.Instance.Files.Player.DeleteAsync(filename);

        Debug.Log($"Successfully deleted design and screenshot for {slotId}");
        return true;
    }
    catch (Exception e)
    {
        Debug.LogError($"Error deleting design {slotId}: {e.Message}");
        return false;
    }
}
```

Figure 6.14. *Deleting Saved Design from Cloud*

The above code deletes a saved design from the cloud by removing both the layout data and its associated screenshot file. This ensures complete cleanup of stored design information, preventing unused data from remaining in the system.

Reloading Designs

```
// Fetch all slots from Cloud
var designsDict = await CloudSaveManager.Instance.GetAllSavedDesignsAsync();

// 🟡 SORTING: Convert to List and sort by Key (Timestamp) in Descending order (Newest first)
List<KeyValuePair<string, SaveData>> designsList = new List<KeyValuePair<string, SaveData>>(designsDict);
designsList.Sort((a, b) => string.Compare(b.Key, a.Key));

foreach (var kvp in designsList)
{
    string currentSlotId = kvp.Key;
    SaveData currentData = kvp.Value;

    // 🟢 Spawn UI Card
    GameObject cardObj = Instantiate(slotCardPrefab, slotsContainer);

    // 🟣 Get the Card Brain
    DesignSlotCard cardBrain = cardObj.GetComponent<DesignSlotCard>();
    if (cardBrain != null)
    {
        cardBrain.Setup(currentSlotId, currentData);
    }
    else
    {
        Debug.LogWarning("Prefab missing DesignSlotCard component! Please attach it to avoid mismatches.");
    }
}

finally
{
    isReloading = false;
}

public void LoadDesignDetails(string slotId, SaveData data)
{
    saveSlotsMenuPanel.SetActive(false);
    // call RoomSetupManager to spawn the group!
    FindObjectOfType<RoomSetupManager>().LoadSavedLayout(data);
}
```

Figure 6.15. *Loading and Displaying Saved Design Slots*

The code retrieves all saved designs from the cloud and organizes them in descending order based on their creation time. It then dynamically generates UI cards for each saved design and assigns the corresponding data. This allows users to view, select, and load previously saved layouts efficiently.

6.3.5 Room Setup and Budget Initialization

Room Selection and UI Panel Activation

```
string selectedRoom = roomDropdown.options[roomDropdown.value].text;

roomPanel.SetActive(false);
HideAllPanels();

switch (selectedRoom)
{
    case "Bedroom":
        bedroomPanel.SetActive(true);
        break;

    case "Living Room":
        livingRoomPanel.SetActive(true);
        break;

    case "Dining":
        diningPanel.SetActive(true);
        break;

    default:
        Debug.LogWarning("Unknown room selected: " + selectedRoom);
        break;
}

if (addMoreButton != null)
    addMoreButton.SetActive(false);

if (costPanel != null)
    costPanel.SetActive(true);

// Tell SaveLoadUI that setup is done
if (SaveLoadUIManager.Instance != null)
{
    SaveLoadUIManager.Instance.OnSetupConfirmed();
}

Debug.Log("Selected Room: " + selectedRoom);
}
```

Figure 6.16. *Room Selection and UI Panel Activation*

The above code activates the appropriate furniture panel based on the selected room type. It hides unnecessary UI elements and displays only the relevant panel, ensuring a structured and user-friendly interface. This allows users to view furniture options specific to the chosen room category.

Furniture Selection and Price Passing from UI

```

void OnViewARClicked()
{
    // CHECK PREFAB
    if (furniturePrefab == null)
    {
        Debug.LogWarning("Furniture Prefab not assigned on " + gameObject.name);
        return;
    }

    // SET SELECTED FURNITURE
    if (FurnitureManager.Instance != null)
    {
        FurnitureManager.Instance.SetFurniture(furniturePrefab);

        // 🔥 UPDATED LOG (IMPORTANT)
        Debug.Log("NEW furniture selected ✅: " + furniturePrefab.name);

        // PASS PRICE TO COST MANAGER
        if (CostManager.Instance != null && furniturePrice != null)
        {
            string priceText = furniturePrice.text;
            // Parse price from text like "Rs. 15000" or "RS. 15000"
            string numStr = System.Text.RegularExpressions.Regex.Replace(priceText, @"[^0-9.]", "");
            float price = 0f;
            float.TryParse(numStr, out price);
            string itemNameStr = furnitureName != null ? furnitureName.text : furniturePrefab.name;
            CostManager.Instance.SetPendingPrice(itemNameStr, price);
        }

        // HIDE PANELS + SHOW ADD MORE
        RoomSetupManager manager = FindObjectOfType<RoomSetupManager>();
        if (manager != null)
        {
            manager.HideAllPanels();

            if (manager.addMoreButton != null)
            {
                manager.addMoreButton.SetActive(true);
            }
        }
    }
    else
    {
        Debug.LogWarning("FurnitureManager instance not found!");
    }
}

```

Figure 6.17. Furniture Selection and Price Passing from UI

The code handles the selection of furniture from the UI by assigning the selected prefab to the system. It also extracts and passes the corresponding price information to the Cost Manager. Additionally, the UI is updated to transition from the selection panel to the AR placement stage.

Budget Parsing and Initialization

```

public void OnConfirmClicked()
{
    // 🚨 FALLBACK: If user forgot to assign budgetInputField, find it dynamically before hiding the panel!
    if (budgetInputField == null)
    {
        TMP_InputField foundField = FindObjectOfType<TMP_InputField>();
        if (foundField != null)
        {
            Debug.Log("🔍 Auto-found TMP InputField: " + foundField.gameObject.name);
            budgetInputField = foundField;
        }
    }

    string incomingBudgetText = "";

    if (budgetInputField != null)
    {
        incomingBudgetText = budgetInputField.text;
    }
    else
    {
        // FALLBACK 2: Check for Legacy InputField if TMP_InputField isn't used
        UnityEngine.UI.InputField legacyField = FindObjectOfType<UnityEngine.UI.InputField>();
        if (legacyField != null)
        {
            Debug.Log("🔍 Auto-found Legacy InputField: " + legacyField.gameObject.name);
            incomingBudgetText = legacyField.text;
        }
    }

    // 🚨 Apply User Budget if provided
    if (!string.IsNullOrEmpty(incomingBudgetText))
    {
        // Strip out any currency letters, commas, or spaces (e.g. "Rs. 50,000" -> "50000")
        string cleanNum = System.Text.RegularExpressions.Regex.Replace(incomingBudgetText, @"[^0-9]", "");

        float newBudget = 0f;
        if (float.TryParse(cleanNum, out newBudget))
        {
            if (CostManager.Instance != null && newBudget > 0)
            {
                CostManager.Instance.maxBudget = newBudget;
                CostManager.Instance.UpdateUI();
                Debug.Log("🔥 Custom Budget Set: " + newBudget);
            }
        }
    }
}

```

Figure 6.18. Budget Input Parsing and Initialization

The above code processes the user-entered budget by extracting numeric values and converting them into a usable format. The validated budget is then assigned to the system, and the UI is updated accordingly. This ensures accurate budget initialization before starting the design process.

7 UI DESIGN

The UI Design of the AR-Space application is engineered to bridge the gap between complex augmented reality technology and intuitive user interaction. By prioritizing a clean, clutter-free workspace, the design ensures the live camera feed remains the focal point, allowing users to visualize 3D furniture models within their physical environment without distraction. The interface utilizes a modular, card-based layout for furniture selection and integrates seamless gesture-based controls for manipulating objects, ensuring a fluid and user-centric design process.

7.1 Room Type Selection and Budget Setup Interface

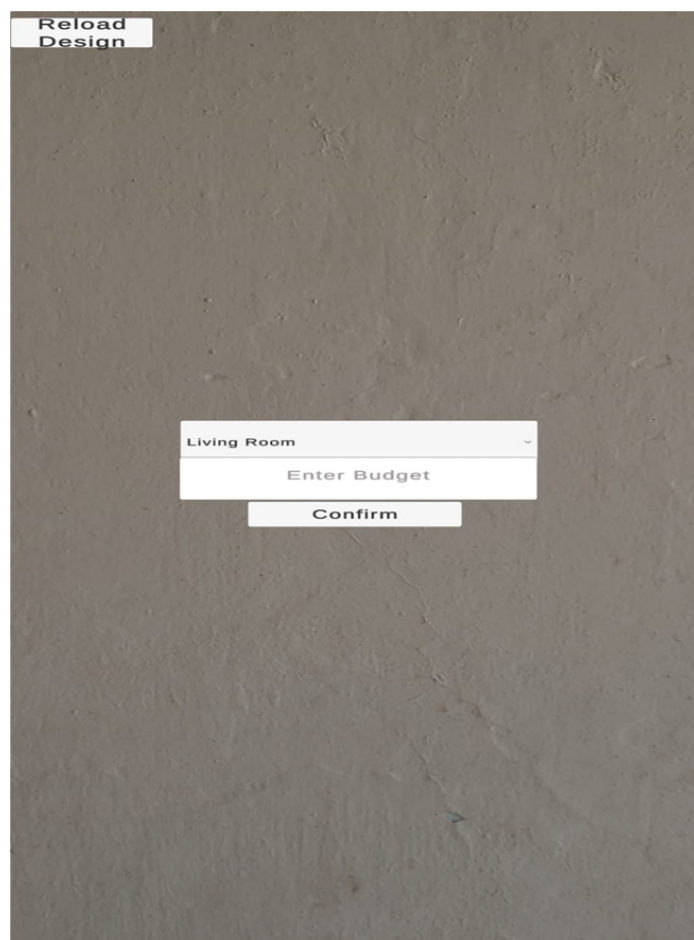


Figure 7.1. *Room Type Selection and Budget Setup Interface*

The above interface represents the initial configuration screen of the application, designed to collect essential user inputs before entering the AR environment. Each component in this interface serves a specific purpose to ensure a smooth and guided user experience.

Reload Design Button: Located at the top-left corner, this button allows users to retrieve previously saved interior layouts from the cloud. It provides quick access to existing designs without requiring users to restart the process.

Room Type Dropdown: This dropdown menu enables users to select the category of the room, such as Living Room, Bedroom, or Dining Room. The selected room type determines the set of furniture options displayed later, ensuring context-aware recommendations. It helps streamline the design process by filtering irrelevant items and presenting only suitable furniture choices for the selected environment.

Budget Input Field: The input field allows users to enter their budget for the interior design. It accepts numerical values and is processed internally to remove unwanted characters (such as currency symbols). This budget is later used for real-time cost tracking and validation. Additionally, it helps guide users in making practical design decisions by preventing selections that exceed the defined budget.

Confirm Button: The confirm button finalizes the user's input. Upon clicking, the system stores the selected room type and budget, hides the setup panel, and activates the AR design workspace along with relevant UI panels. This action serves as a transition point from configuration to interaction, ensuring that all user inputs are properly initialized before entering the design phase.

Background AR Camera View: The semi-transparent background represents the live camera feed from the device. This provides users with a real-world context even before entering the full AR interaction mode. It enhances user understanding of spatial alignment and prepares them for seamless interaction with virtual objects in the actual environment.

7.2 Furniture Selection Panel

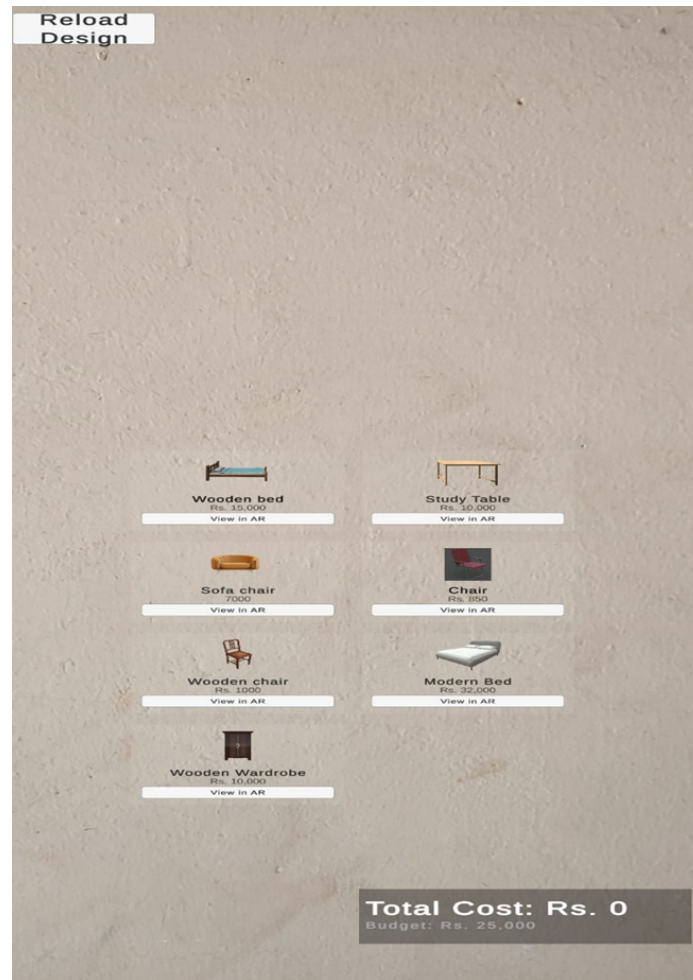


Figure 7.2. Furniture Recommendation Panel

The above interface represents the furniture selection panel, where users can browse and choose items to place in the AR environment. The design follows a card-based layout, making it visually organized and easy to navigate. The items are arranged in a grid format, improving readability and ensuring efficient use of screen space, especially on mobile devices.

Furniture Item Cards: Each furniture item is presented in the form of an individual card, which includes a preview image, item name, and price details. The use of images provides a realistic visual reference, helping users understand the appearance and style of the furniture before placing it in the AR scene. The consistent structure of these cards ensures uniformity across the interface, making it easier for users to compare multiple items side by side. This card-based representation also improves accessibility and reduces cognitive load during selection.

Price Display: The price of each furniture item is clearly displayed below the item name within the card. This feature enables users to make cost-aware decisions while browsing

through available options. By providing transparent pricing information upfront, the interface supports better financial planning and prevents unexpected cost accumulation during the design process.

“View in AR” Button: Each furniture card includes a dedicated “View in AR” button, which acts as the primary interaction control for selecting an item. When this button is clicked, the corresponding furniture prefab is activated and loaded into the AR environment, allowing the user to place it within their real-world space. This functionality ensures a seamless transition between selection and visualization, improving the interactivity of the application.

Reload Design Button: It is positioned at the top-left corner of the interface. This feature allows users to exit the current design session and load previously saved layouts from the cloud. It enhances usability by enabling quick access to saved designs, thereby supporting continuity and flexibility in the design process.

Real-Time Cost Display Panel: At the bottom of the interface, a real-time cost display panel provides users with an overview of the total cost of all selected furniture items. This panel also displays the user-defined budget, allowing users to monitor their spending in real time. The dynamic updating of cost values ensures that users are always aware of their current financial status while designing.

Budget Feedback Mechanism: The interface incorporates a visual feedback system to assist users in maintaining their budget constraints. When the total cost exceeds the predefined budget, warning indicators such as color changes (e.g., red highlights) are triggered. This immediate feedback helps users identify overspending and make necessary adjustments, thereby improving decision-making and enhancing the overall user experience.

7.3 AR Design Workspace Interface



Figure 7.3. *AR Design Workspace Interface*

The above interface represents the core working environment of the application, where augmented reality is used to visualize and interact with virtual furniture in real-world space. The design focuses on minimizing UI clutter while providing essential controls for seamless interaction.

AR Camera View (Real-World Background): The background of the interface is the live camera feed captured from the mobile device. This enables the system to detect real-world surfaces such as floors using AR plane detection. The continuous camera input ensures that the virtual objects remain anchored to the environment even when the user moves the device, providing a stable and realistic AR experience.

Virtual Furniture Placement (3D Object Rendering): The placed furniture object is rendered on the detected surface using raycasting techniques. The object is positioned based on the intersection of user touch input and detected planes. It maintains proper scale, orientation, and perspective, ensuring that it blends naturally with the real environment.

Save Design Button: This button allows users to store the current interior layout. When

clicked, the system captures all necessary data, including object positions, rotations, room type, and budget. Additionally, a screenshot of the current scene is taken and saved along with the layout in the cloud, enabling easy identification during reload.

Reload Design Button: Located at the top-left, this button provides access to previously saved layouts. Users can reload designs at any stage of interaction, allowing them to continue editing or compare different configurations without losing progress.

“Add More” Button: This button reopens the furniture selection panel without exiting the AR environment. It enables users to add additional furniture items dynamically, supporting an iterative design process where users can gradually build their layout.

Real-Time Cost Display Panel: Positioned at the bottom of the screen, this panel continuously displays the total cost of all placed furniture items along with the predefined budget. The system updates this value instantly whenever an item is added or removed, ensuring real-time feedback.

Budget Feedback Mechanism: The cost panel also provides visual indicators when the total cost exceeds the defined budget. This may include color changes or warning messages, helping users maintain financial control while designing.

Gesture-Based Interaction System: The interface supports multiple touch-based interactions for intuitive object manipulation. A single tap on a furniture object is used to select it. Once selected, the user can drag (touch and move) the screen to reposition the object across the detected surface. A two-finger gesture is used to rotate the object, where the angle between the fingers determines the rotation direction and degree. Additionally, a double-tap gesture on a selected object triggers deletion, removing it from the scene while also updating the cost system. These gesture-based controls eliminate the need for extra UI buttons and provide a natural and user-friendly interaction experience.

7.4 Saved Design Reload Interface

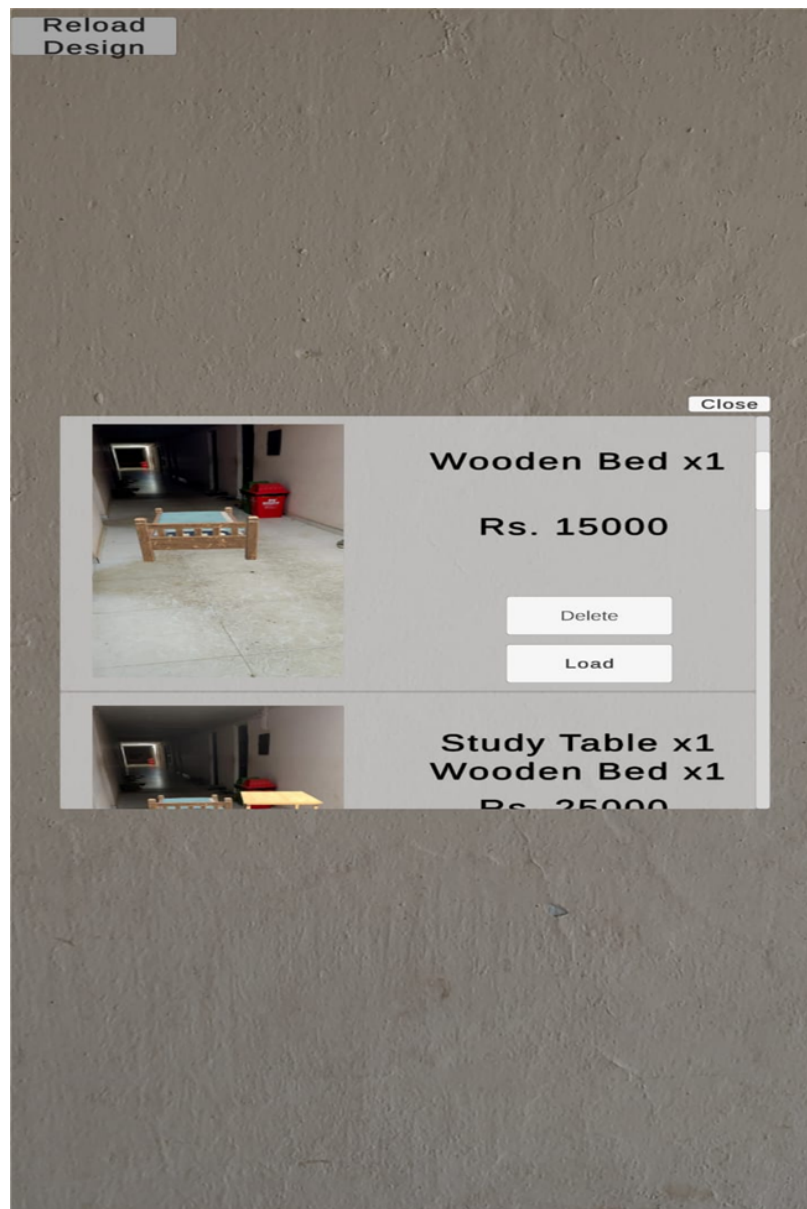


Figure 7.4. *Saved Design Reload Interface*

The above interface represents the design management panel used for saving, viewing, loading, and deleting previously created interior layouts. This UI plays a crucial role in providing persistence and flexibility to the user. The interface supports scrolling, allowing users to navigate through multiple saved designs. This ensures scalability when handling a large number of saved layouts. The panel supports vertical scrolling, allowing multiple saved designs to be displayed within a limited screen space. This ensures smooth navigation even when the number of saved layouts increases.

Saved Design Cards: Each saved layout is represented as a card containing structured information about that particular design. The card groups all relevant details such as preview,

item list, and cost in a single unit, making it easy for users to scan and compare multiple designs.

Screenshot Preview (Visual Identification): Each card includes a thumbnail image captured at the time of saving the design. This screenshot reflects the actual AR scene, helping users visually recall the layout. This reduces reliance on text-based identification and improves usability.

Furniture Summary (Item Name and Quantity): The interface displays the list of furniture items included in the design along with their count (e.g., “Sofa Chair ×1”). This allows users to understand the composition of the layout at a glance without loading it.

Cost Information Display: The total cost associated with the saved design is clearly presented. This helps users compare different layouts based on financial constraints and choose the most suitable option.

Load Button (Design Reconstruction): The “Load” button allows users to restore a selected design into the AR environment. When activated, the system retrieves stored data from the cloud, reconstructs the spatial layout, repositions all furniture objects, and restores associated parameters such as budget and room category.

Delete Button (Data Management): The “Delete” button enables users to permanently remove a selected design from cloud storage. This includes both layout data and associated screenshot files. It helps maintain storage efficiency and allows users to manage unwanted designs.

Close Button (Exit Control): The “Close” button allows users to exit the panel and return to the AR workspace. It ensures that users can easily switch back to design mode without performing any additional actions.

8 TESTING

The testing phase plays a crucial role in ensuring the reliability, accuracy, and usability of the AR Interior Design application. The system was evaluated under different conditions to verify that all modules function correctly and interact seamlessly. Testing was carried out both in the Unity Editor and on real Android devices to ensure consistency between development and real-world usage. The primary objective was to identify and eliminate errors, improve performance, and validate that the application meets user requirements effectively.

8.1 Functional Testing

Functional testing was conducted to verify that each feature of the application performs according to the specified requirements. Every module was tested individually and in integration with other components to ensure proper data flow and functionality.

Key functionalities tested include:

- **AR Object Placement:** Ensured that furniture models are correctly placed on detected planes using AR technology.
- **Furniture Selection System:** Verified that selecting a furniture item from the UI correctly loads the corresponding 3D model.
- **Cost Estimation System:** Checked whether the total cost updates dynamically when items are added or removed.
- **Save and Load Feature:** Validated that designs are stored and retrieved accurately from cloud storage.
- **Room Selection:** Ensured that selecting different room categories updates the available furniture options.

All functional components were tested with multiple inputs to ensure robustness and error-free operation.

8.2 Performance Testing

Performance testing was carried out to evaluate the efficiency and responsiveness of the application under real-time conditions. Since AR applications are resource-intensive, it was important to ensure smooth rendering and minimal lag.

The application was tested for:

- Frame rate stability during AR rendering
- Loading time of 3D models
- Memory usage and optimization
- Responsiveness during multiple object placements

The results indicate that the application performs efficiently without significant delays or crashes, even when multiple objects are present in the scene.

8.3 Gesture-Based Interaction Testing

Gesture testing was conducted to ensure smooth and intuitive manipulation of virtual objects within the AR environment. The application supports multiple touch gestures, which were tested for accuracy and responsiveness.

The gestures and their functionalities are:

- **Tap Gesture:** Used to place furniture objects on detected surfaces.
- **Drag Gesture:** Allows users to move placed objects across the floor.
- **Rotate Gesture:** Allows rotation of objects for proper alignment.

Each gesture was tested multiple times to ensure precise control and a natural user interaction experience.

9 RESULTS AND DISCUSSION

The AR Interior Design application was successfully developed and evaluated to assess its functionality, usability, and performance. The results demonstrate that the system effectively integrates augmented reality with an interactive user interface to provide a practical interior design solution.

During testing, the AR module accurately detected horizontal surfaces and allowed users to place virtual furniture objects in real-world environments. The placement was stable, and objects remained properly anchored, providing a realistic visualization experience. The gesture-based interaction system enabled users to manipulate objects using touch inputs such as tap, drag, and rotate, improving ease of use.

The furniture selection interface functioned efficiently, allowing users to browse and choose items from different categories. Each selected item was correctly displayed in the AR scene along with its name and price. The cost estimation system dynamically updated the total cost as items were added or removed and also provided feedback when the budget limit was exceeded.

The save and load functionality worked reliably through cloud integration, enabling users to store and retrieve their design layouts along with relevant details such as furniture items and total cost. Saved designs were restored accurately, maintaining consistency with the original setup.

From a performance perspective, the application operated smoothly on Android devices with stable rendering and responsive user interactions. Minor issues such as slight sensitivity and slight delay in plane detection were observed but did not significantly affect usability.

Overall, the system achieved its intended objectives by delivering an intuitive and efficient AR-based interior design experience. The integration of real-time visualization, cost estimation, and cloud storage enhances the practicality of the application and demonstrates the potential of augmented reality in interior design planning.

10 GIT HISTORY

```
commit cad0f7e51feed8b3b6009b8a732cc245b13fd289 (HEAD -> master)
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:57:25 2026 +0530

    Improved UI and fixed bugs

commit f22b79f3fca6313dafbb9e11f49a2b9f09f227f6
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:57:01 2026 +0530

    Added save and load design feature

commit cd6d093e5ffeb4ae7aa916f9b87b3f1889f2a8fe
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:53:11 2026 +0530

    Implemented cost estimation system with budget tracking

commit 78011e1e1929cdfb36e72a9d858105d8c02e9fee
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:52:35 2026 +0530

    Integrated gesture-based interaction (move, rotate, scale)

commit 68b72d876657fd8da2233891f5c83a9e9d1d544c
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:39:38 2026 +0530

    Implemented AR plane detection and object placement

commit 00961de1ac32be9f57966ac053a45fd57554a906
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:36:59 2026 +0530

    Project setup and initial configuration

commit 9124d56cbb1d4393a3e25f19fbd485cd79281902
Author: Anoop Krishnan <anoopthadiyoor123@gmail.com>
Date: Tue Apr 14 09:19:54 2026 +0530

    Initial commit
```

Figure 10.1. *Git history*

11 CONCLUSION

The AR Interior Design application was successfully developed to provide an interactive and efficient solution for visualizing interior layouts using augmented reality. The system enables users to place and manipulate virtual furniture in real-world environments, offering a realistic and practical design experience. Key features such as gesture-based interaction, real-time cost estimation, and cloud-based save and load functionality enhance both usability and functionality.

The application effectively addresses the limitations of traditional interior design methods by allowing users to visualize layouts before implementation, thereby improving decision-making and reducing errors. The integration of AR technology with a simple and user-friendly interface ensures accessibility for a wide range of users.

Overall, the project demonstrates the potential of augmented reality in transforming interior design processes by combining visualization, interaction, and data management into a single platform. The system is reliable, efficient, and can be further extended with advanced features in the future.

12 FUTURE WORK

The developed AR Interior Design application provides a solid foundation for interactive and real-time interior visualization. However, there are several areas where the system can be further enhanced to improve functionality, accuracy, and user experience.

One major improvement would be the integration of AI-based furniture recommendation systems, which can suggest suitable furniture based on room type, user preferences, and available space. This would make the application more intelligent and assist users in making better design decisions.

Another enhancement is the incorporation of LiDAR or advanced depth sensing technologies for more accurate surface detection and spatial mapping. This would improve object placement precision and reduce issues related to surface detection and environmental limitations.

The system can also be extended to support multi-user or collaborative design, where multiple users can work on the same layout in real time. This would be particularly useful for professional interior designers and team-based projects.

Additionally, implementing scaling functionality for furniture objects would allow users to adjust the size of items according to their space requirements, increasing flexibility in design.

Further improvements can include enhanced 3D models and realistic rendering techniques, such as better lighting, shadows, and textures, to provide a more immersive experience.

Finally, the application can be expanded to support cross-platform compatibility, including iOS devices and web-based AR, making it accessible to a wider audience.

Overall, these enhancements can significantly improve the system's capabilities and make it a more powerful and versatile interior design tool.

13 APPENDIX

13.1 Minimum Software Requirements

The following software components are required for the development and execution of the AR Interior Design application:

- Operating System: Windows 10 or later
- Development Platform: Unity 2022.3 (or later)
- Programming Language: C#
- AR Framework: AR Foundation
- SDK: ARCore SDK (for Android)
- IDE / Editor: Visual Studio / Visual Studio Code
- Cloud Services: Unity Cloud Services (for data storage)
- Android Build Support: Installed via Unity Hub
- Android OS Version: Android 8.0 (Oreo) or higher

13.2 Minimum Hardware Requirements

For Development (PC/Laptop):

- Processor: Intel i5 or equivalent
- RAM: Minimum 8 GB
- Storage: At least 10 GB free space
- Graphics: Integrated or dedicated GPU (OpenGL 3.0 support or higher)

For Mobile Device (AR Execution):

- Device Type: ARCore-supported Android smartphone
- Processor: Snapdragon 845 / equivalent or higher
- RAM: Minimum 4 GB
- Camera: Required for AR functionality
- Sensors: Gyroscope and Accelerometer
- Storage: Minimum 200 MB free space

14 REFERENCES

- [1] Cuevas JRG, et al. LayOut Loud: an AI powered augmented reality and mobile application for room interior design and layout optimization.,” *In:Proceedings of the International Conference on Intelligent Cybernetics Technology Applications (ICICyTA)2024; Bali, Indonesia. p. 888–894. doi:10.1109/ICICYTA64807.2024.10912928*, Available: <https://ieeexplore.ieee.org/document/10912928>
- [2] TR S, Santhosh A, Ajith A, Gopi A. Interactive interior design using augmented reality and 3D modeling.” *In: Proceedings of the 2nd International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI); 2024; Coimbatore, India. p. 1056–1062. doi:10.1109/ICoICI62503.2024.10696390*, pp. 372–377, 2024. Available: https://www.researchgate.net/publication/384649708_Interactive_Interior_Design_Using_Augmented_Reality_and_3D_Modeling
- [3] R NR, M R, S RB, Sultana S, Nadig NM. Markerless augmented reality application for interior designing.” *n: Proceedings of the 2nd International Conference on Advanced Technologies in Intelligent Control, Environment, Computing and Communication Engineering(ICATIECE); 2022; Bangalore, India. p. 1–5. doi:10.1109/ICATIECE56365.2022.10047281*, Available: <https://ieeexplore.ieee.org/document/10047281>
- [4] Git Documentation, “Git Reference Manual” Available: <https://git-scm.com/docs>
- [5] Atlassian, “Git Tutorials and Training.” Available: <https://www.atlassian.com/git/tutorials>
- [6] Google Developer, “ARCore Developer Guide” Available: <https://developers.google.com/ar>
- [7] Unity Technologies, “AR Foundation Documentation.” Available: <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation>
- [8] Unity Technologies, “Unity Hub Documentation.” Available: <https://docs.unity3d.com/hub/manual/>